

Nombres: Juan José Jaramillo Mora y Sebastián Giraldo Franco

Actividad AWS

1. AWS Secrets Manager

Investigue qué es AWS Secrets Manager, qué tipo de información permite guardar y cómo se controla quién puede leer un secreto. Distinga entre crear un secreto, recuperarlo desde una aplicación y rotarlo.

R/ AWS Secrets Manager es un servicio administrado de AWS diseñado para almacenar, administrar, recuperar y rotar información sensible, evitando que las credenciales queden escritas directamente dentro del código fuente de una aplicación. Permite almacenar información como usuarios y contraseñas de bases de datos, credenciales de aplicaciones, tokens OAuth, claves API y otros valores sensibles. El acceso a los secretos se controla principalmente mediante AWS Identity and Access Management (IAM). Las políticas IAM permiten especificar qué usuario, rol o aplicación puede realizar determinadas acciones sobre un secreto específico.

Operación	Descripción
Crear un secreto	Consiste en registrar por primera vez la información sensible dentro de Secrets Manager. Se define un nombre, el valor del secreto y la clave de cifrado utilizada.
Recuperar un secreto	Una aplicación autenticada en AWS solicita el valor durante su ejecución. La contraseña no necesita quedar almacenada en el código fuente.
Rotar un secreto	Consiste en sustituir periódicamente el valor actual por uno nuevo. Secrets Manager permite configurar rotación automática para determinados tipos de secretos y servicios

¿Cómo se cobran los secretos almacenados y las llamadas a la API?

R/ AWS Secrets Manager cobra por dos conceptos principales: por cada secreto almacenado y por las llamadas realizadas a la API. Como referencia, el almacenamiento tiene un costo aproximado de USD 0,40 por secreto al mes, y las llamadas a la API tienen un costo aproximado de USD 0,05 por cada 10.000 solicitudes.

¿Qué ocurre con el costo si el programa consulta el mismo secreto repetidamente?

R/ Cada llamada a GetSecretValue cuenta como una solicitud a la API. Por lo tanto, si el programa consulta el mismo secreto muchas veces, aumenta el número de llamadas y, con

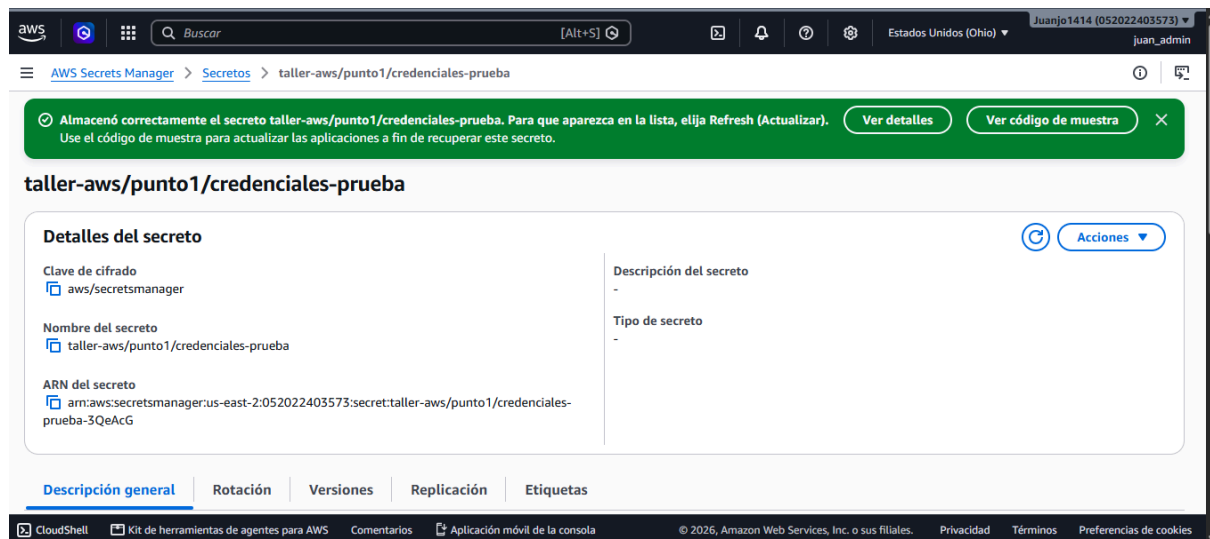
ello, el costo asociado a la API. Además del costo, realizar consultas repetidas puede incrementar la latencia de la aplicación.

¿Cuándo convendría guardar temporalmente el valor en memoria?

R/ Conviene almacenar temporalmente el secreto en memoria cuando una aplicación necesita utilizar repetidamente la misma credencial y esta no cambia constantemente. Esto permite reducir el número de llamadas a Secrets Manager, disminuir la latencia y reducir costos. Sin embargo, el valor no debería mantenerse en memoria indefinidamente; es recomendable utilizar un tiempo de expiración para que la aplicación consulte nuevamente Secrets Manager y pueda obtener una versión actualizada en caso de que el secreto haya cambiado o rotado.

Entregables:

- **Código Python:** Está en github.
- **Política de acceso:** Está en github.
- **Explicación de la actualización:** el valor de la contraseña se modificó directamente en AWS Secrets Manager y el mismo programa Python, sin cambios en el código, recuperó nuevamente la versión actual.
- **Estimación de costos:** el costo de la práctica fue mínimo debido al corto tiempo de almacenamiento del secreto y al bajo número de llamadas a la API.



taller-aws/secret-prueba

This secret has been scheduled for deletion.

Secret details



Cancel deletion

Encryption key

aws/secretsmanager

Secret name

taller-aws/secret-prueba

Secret ARN

arn:aws:secretsmanager:us-east-2:211107604485:secret:taller-aws/secret-prueba-eQsmc6

Secret description

Secreto de prueba para la actividad AWS - Punto 1

Secret type

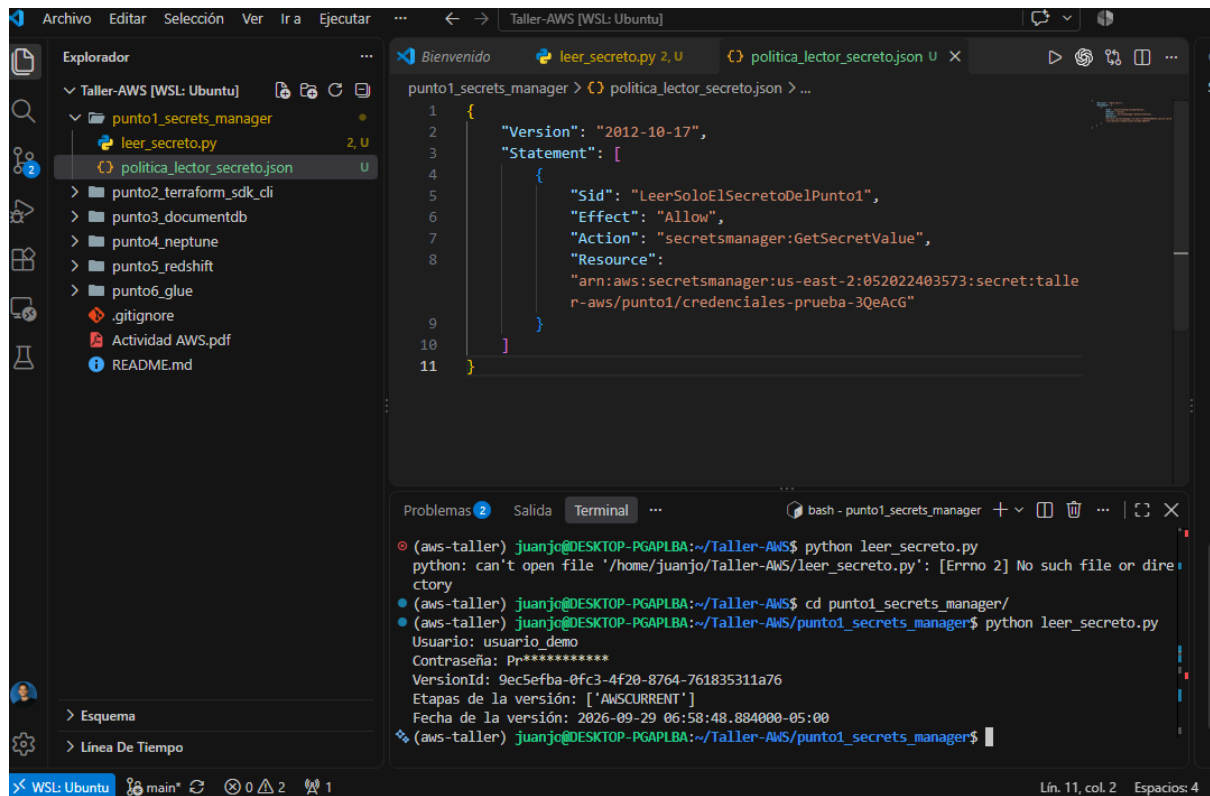
-

Deleted on

29/9/2026

```
(aws-taller) juanjo@DESKTOP-PGAPLBA:~$  
(aws-taller) juanjo@DESKTOP-PGAPLBA:~$ aws configure --profile lector-secreto  
  
Tip: You can deliver temporary credentials to the AWS CLI using your AWS Console session by running the command 'aws log  
in'.  
  
AWS Access Key ID [None]: AKIAQYHGG4H2QZ6BLZP7  
AWS Secret Access Key [None]: I5fy/ZKbShdSTaXDyVRVf7uEvFeGRn+Dw2HVeMPs  
Default region name [None]: us-east-2  
Default output format [None]: json  
  
aws: [WARNING]: The file '/home/juanjo/.aws/credentials' is accessible by other users. Consider running 'chmod 600 /home  
/juanjo/.aws/credentials' to restrict access to only your user.  
(aws-taller) juanjo@DESKTOP-PGAPLBA:~$ chmod 600 /home/juanjo/.aws/credentials  
(aws-taller) juanjo@DESKTOP-PGAPLBA:~$ |
```

```
juanjo@DESKTOP-PGAPLBA: ~ x Ubuntu x + v  
(base) juanjo@DESKTOP-PGAPLBA:~$  
(base) juanjo@DESKTOP-PGAPLBA:~$ aws sts get-caller-identity --profile lector-secreto  
{  
  "UserId": "AIDAQYHGG4H2QTZ2KLSIY",  
  "Account": "052022403573",  
  "Arn": "arn:aws:iam::052022403573:user/app_lector_secreto"  
}  
(base) juanjo@DESKTOP-PGAPLBA:~$ aws secretsmanager list-secrets --profile lector-secreto  
  
aws: [ERROR]: An error occurred (AccessDeniedException) when calling the ListSecrets operation: User: arn:aw  
s:iam::052022403573:user/app_lector_secreto is not authorized to perform: secretsmanager:ListSecrets because  
no identity-based policy allows the secretsmanager:ListSecrets action  
(base) juanjo@DESKTOP-PGAPLBA:~$
```

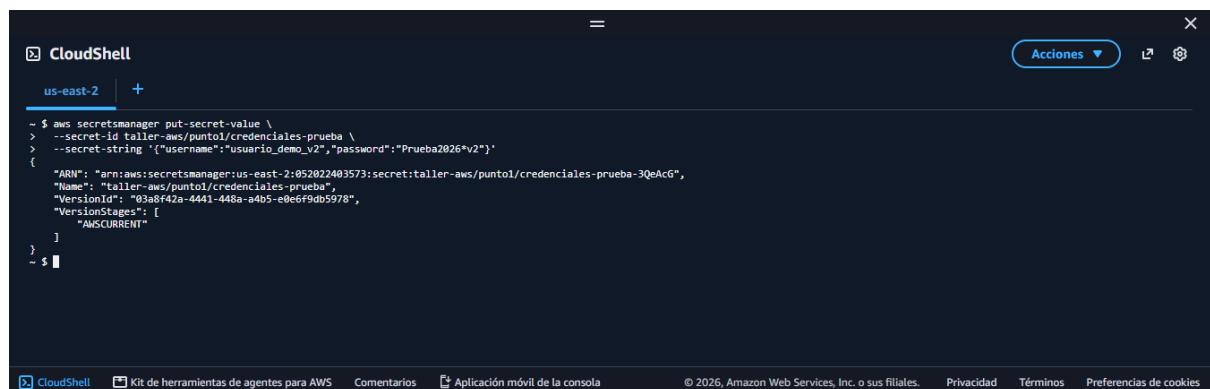


The image shows a VS Code editor window titled 'Taller-AWS [WSL: Ubuntu]'. The Explorer sidebar on the left shows a project structure with folders like 'punto1_secrets_manager', 'punto2_terraform_sdk_cli', 'punto3_documentdb', 'punto4_neptune', 'punto5_redshift', 'punto6_glue', and files like 'leer_secreto.py', 'politica_lector_secreto.json', '.gitignore', 'Actividad AWS.pdf', and 'README.md'. The main editor area shows the 'politica_lector_secreto.json' file with the following content:

```
1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Sid": "LeerSoloElSecretoDelPunto1",
6       "Effect": "Allow",
7       "Action": "secretsmanager:GetSecretValue",
8       "Resource": "arn:aws:secretsmanager:us-east-2:052022403573:secret:taller-aws/punto1/credenciales-prueba-3QeAcG"
9     }
10  ]
11 }
```

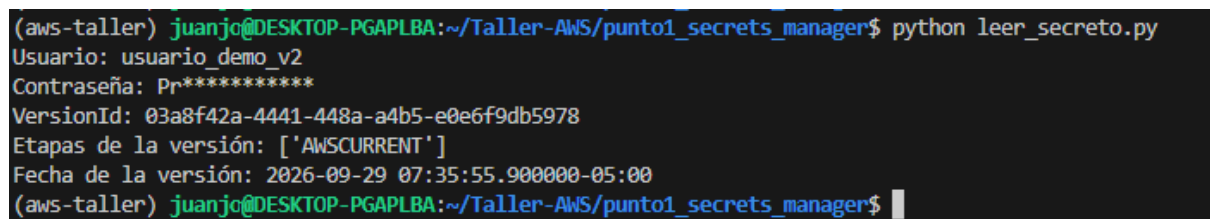
Below the editor is a terminal window with the following output:

```
(aws-taller) juanjoc@DESKTOP-PGAPLBA:~/Taller-AWS$ python leer_secreto.py
python: can't open file '/home/juanjo/Taller-AWS/leer_secreto.py': [Errno 2] No such file or directory
(aws-taller) juanjoc@DESKTOP-PGAPLBA:~/Taller-AWS$ cd punto1_secrets_manager/
(aws-taller) juanjoc@DESKTOP-PGAPLBA:~/Taller-AWS/punto1_secrets_manager$ python leer_secreto.py
Usuario: usuario_demo
Contraseña: Pr*****
VersionId: 9ec5efba-0fc3-4f20-8764-761835311a76
Etapas de la versión: ['AWSCURRENT']
Fecha de la versión: 2026-09-29 06:58:48.884000-05:00
(aws-taller) juanjoc@DESKTOP-PGAPLBA:~/Taller-AWS/punto1_secrets_manager$
```



The image shows an AWS CloudShell terminal window. The command entered is:

```
$ aws secretsmanager put-secret-value \
> --secret-id taller-aws/punto1/credenciales-prueba \
> --secret-string '{"username":"usuario_demo_v2","password":"Prueba2026*v2"}'
{
  "ARN": "arn:aws:secretsmanager:us-east-2:052022403573:secret:taller-aws/punto1/credenciales-prueba-3QeAcG",
  "Name": "taller-aws/punto1/credenciales-prueba",
  "VersionId": "03a8f42a-4441-448a-a4b5-e0e6f9db5978",
  "VersionStages": [
    "AWSCURRENT"
  ]
}
```



The image shows a terminal window with the following output:

```
(aws-taller) juanjoc@DESKTOP-PGAPLBA:~/Taller-AWS/punto1_secrets_manager$ python leer_secreto.py
Usuario: usuario_demo_v2
Contraseña: Pr*****
VersionId: 03a8f42a-4441-448a-a4b5-e0e6f9db5978
Etapas de la versión: ['AWSCURRENT']
Fecha de la versión: 2026-09-29 07:35:55.900000-05:00
(aws-taller) juanjoc@DESKTOP-PGAPLBA:~/Taller-AWS/punto1_secrets_manager$
```

CloudShell

us-east-2



```
{
  "Versions": [
    {
      "VersionId": "9ec5efba-0fc3-4f20-8764-761835311a76",
      "VersionStages": [
        "AWSPREVIOUS"
      ],
      "LastAccessedDate": "2026-09-29T00:00:00+00:00",
      "CreatedDate": "2026-09-29T11:58:48.884000+00:00",
      "KmsKeyIds": [
        "DefaultEncryptionKey"
      ]
    },
    {
      "VersionId": "03a8f42a-4441-448a-a4b5-e0e6f9db5978",
      "VersionStages": [
        "AWSCURRENT"
      ]
    }
  ],
  "CreatedDate": "2026-09-29T11:58:48.884000+00:00"
}
```

- Evidencia de eliminación:

```
- $ aws secretsmanager describe-secret --secret-id taller-aws/punto1/credenciales-prueba
{
  "ARN": "arn:aws:secretsmanager:us-east-2:852022403573:secret:taller-aws/punto1/credenciales-prueba-3QeAcG",
  "Name": "taller-aws/punto1/credenciales-prueba",
  "LastChangedDate": "2026-09-29T12:35:55.904000+00:00",
  "LastAccessedDate": "2026-09-29T00:00:00+00:00",
  "Tags": [],
  "VersionIdsToStages": {
    "03a8f42a-4441-448a-a4b5-e0e6f9db5978": [
      "AWSCURRENT"
    ],
    "9ec5efba-0fc3-4f20-8764-761835311a76": [
      "AWSPREVIOUS"
    ]
  },
  "CreatedDate": "2026-09-29T11:58:48.684000+00:00"
}
- $
- $ aws secretsmanager delete-secret \
> --secret-id taller-aws/punto1/credenciales-prueba \
> --force-delete-without-recovery
{
  "ARN": "arn:aws:secretsmanager:us-east-2:852022403573:secret:taller-aws/punto1/credenciales-prueba-3QeAcG",
  "Name": "taller-aws/punto1/credenciales-prueba",
  "DeletionDate": "2026-09-29T12:39:20.486000+00:00"
}
```

```
    "03a8f42a-4441-448a-a4b5-e0e6f9db5978": [
      "AWSCURRENT"
    ],
    "9ec5efba-0fc3-4f20-8764-761835311a76": [
      "AWSPREVIOUS"
    ]
  },
  "CreatedDate": "2026-09-29T11:58:48.684000+00:00"
}
- $
- $ aws secretsmanager delete-secret \
> --secret-id taller-aws/punto1/credenciales-prueba \
> --force-delete-without-recovery
{
  "ARN": "arn:aws:secretsmanager:us-east-2:852022403573:secret:taller-aws/punto1/credenciales-prueba-3QeAcG",
  "Name": "taller-aws/punto1/credenciales-prueba",
  "DeletionDate": "2026-09-29T12:39:20.486000+00:00"
}
- $
- $ aws secretsmanager describe-secret --secret-id taller-aws/punto1/credenciales-prueba
aws: [ERROR]: An error occurred (ResourceNotFoundException) when calling the DescribeSecret operation: Secrets Manager can't find the specified secret.
- $
```

```
(aws-taller) juanj@DESKTOP-PGAPLBA:~/Taller-AWS/punto1_secrets_manager$ python leer_secreto.py
No se pudo leer el secreto: UnrecognizedClientException
(aws-taller) juanj@DESKTOP-PGAPLBA:~/Taller-AWS/punto1_secrets_manager$
```

2. Despliegue con Terraform, SDK y AWS CLI

Investigue qué es Terraform y cómo se utiliza para definir, planificar, crear y eliminar infraestructura.

R/ Terraform es una herramienta de Infraestructura como Código (Infrastructure as Code, IaC) desarrollada por HashiCorp. Permite definir y administrar infraestructura tecnológica mediante archivos de configuración, en lugar de crear los recursos manualmente desde la consola de un proveedor cloud. Con Terraform es posible describir recursos como redes, máquinas virtuales, bases de datos, almacenamiento y otros servicios de AWS de manera reproducible y versionable. Terraform trabaja de forma declarativa: el usuario define el estado deseado de la infraestructura y la herramienta determina qué acciones son necesarias para alcanzar ese estado. Para comunicarse con AWS utiliza el AWS Provider, que traduce la configuración de Terraform en llamadas a las APIs de AWS. Su flujo básico es:

- **terraform init:** inicializa el proyecto y descarga los proveedores.
- **terraform plan:** muestra qué cambios se van a realizar.
- **terraform apply:** crea o modifica la infraestructura.
- **terraform destroy:** elimina los recursos creados.
-

Investigue y documente los permisos que necesita para crear y eliminar un bucket de S3, una instancia EC2, una instancia RDS y los componentes de red utilizados.

R/ El usuario IAM user_cli se configura con credenciales de acceso programático para que pueda autenticarse desde AWS CLI, Boto3 y Terraform, evitando usar la cuenta raíz. Los permisos necesarios dependen de los recursos que se crearán y eliminarán:

- **S3:** crear/eliminar buckets y consultar su estado.
- **EC2:** crear/terminar instancias y consultar imágenes, tipos y estado.
- **RDS:** crear/eliminar instancias y consultar su configuración.
- **Red:** crear/eliminar VPC, subredes, tablas de rutas, Internet Gateway y Security Groups, además de permisos Describe*.

Despliegue con Terraform

- **Evidencia 1 – Terraform Apply exitoso**

```

aws_db_instance.taller: Still creating... [03m34s elapsed]
aws_db_instance.taller: Still creating... [03m45s elapsed]
aws_db_instance.taller: Still creating... [03m55s elapsed]
aws_db_instance.taller: Still creating... [04m05s elapsed]
aws_db_instance.taller: Still creating... [04m15s elapsed]
aws_db_instance.taller: Still creating... [04m25s elapsed]
aws_db_instance.taller: Still creating... [04m35s elapsed]
aws_db_instance.taller: Still creating... [04m45s elapsed]
aws_db_instance.taller: Still creating... [04m56s elapsed]
aws_db_instance.taller: Still creating... [05m06s elapsed]
aws_db_instance.taller: Creation complete after 5m6s [id=db-6CRQOK3YTAJYX3BY6AV4STPY2M]
Apply complete! Resources: 8 added, 0 changed, 0 destroyed.

```

- Evidencia 2 – Bucket S3 creado con Terraform

```

(aws-taller) eia2024@PC:~/Taller-AWS/punto2_terraform_sdk_cli/terraform$ aws s3 ls --profile user_cli
2026-09-22 06:16:28 sgf-demo-bucket
2026-09-29 09:48:35 taller-aws-p2-211107604485-tf

```

- Evidencia 3 – Instancia EC2 creada con Terraform

```

(aws-taller) eia2024@PC:~/Taller-AWS/punto2_terraform_sdk_cli/terraform$ aws ec2 describe-instances \
--region us-east-2 \
--profile user_cli \
--query "Reservations[0].Instances[0].{ID:InstanceId,State:State.Name,Type:InstanceType}"
[
  {
    "ID": "i-0ff92537236809fc7",
    "State": "running",
    "Type": "t3.micro"
  }
]

```

- Evidencia 4 – Instancia RDS creada con Terraform

```

(aws-taller) eia2024@PC:~/Taller-AWS/punto2_terraform_sdk_cli/terraform$ aws rds describe-db-instances \
--region us-east-2 \
--profile user_cli \
--query "DBInstances[0].{ID:DBInstanceIdentifier,Status:DBInstanceStatus,Class:DBInstanceClass,Engine:Engine}"
[
  {
    "ID": "taller-punto2-rds",
    "Status": "available",
    "Class": "db.t3.micro",
    "Engine": "mysql"
  }
]

```

- Evidencia 5 – Contraseña de RDS administrada por Secrets Manager

```

(aws-taller) eia2024@PC:~/Taller-AWS/punto2_terraform_sdk_cli/terraform$ aws rds describe-db-instances \
--db-instance-identifier taller-punto2-rds \
--region us-east-2 \
--profile user_cli \
--query "DBInstances[0].MasterUserSecret"
{
  "SecretArn": "arn:aws:secretsmanager:us-east-2:211107604485:secret:rds!db-6cac5212-b0de-45bf-b670-a69e4e
ea3f46-u0LT6M",
  "SecretStatus": "active",
  "KmsKeyId": "arn:aws:kms:us-east-2:211107604485:key/91775893-f4b9-4121-80a1-17e632ba8549"
}
(aws-taller) eia2024@PC:~/Taller-AWS/punto2_terraform_sdk_cli/terraform$

```

- Evidencia 6 – Eliminación de recursos con Terraform

```
aws_instance.taller: Destruction complete after 52s
aws_db_instance.taller: Still destroying... [id=db-6CRQOK3YTAJ3BY6AV4STPY2M, 00m41s elapsed]
aws_db_instance.taller: Still destroying... [id=db-6CRQOK3YTAJ3BY6AV4STPY2M, 00m51s elapsed]
aws_db_instance.taller: Still destroying... [id=db-6CRQOK3YTAJ3BY6AV4STPY2M, 01m02s elapsed]
aws_db_instance.taller: Still destroying... [id=db-6CRQOK3YTAJ3BY6AV4STPY2M, 01m12s elapsed]
aws_db_instance.taller: Still destroying... [id=db-6CRQOK3YTAJ3BY6AV4STPY2M, 01m22s elapsed]
aws_db_instance.taller: Still destroying... [id=db-6CRQOK3YTAJ3BY6AV4STPY2M, 01m32s elapsed]
aws_db_instance.taller: Still destroying... [id=db-6CRQOK3YTAJ3BY6AV4STPY2M, 01m42s elapsed]
aws_db_instance.taller: Still destroying... [id=db-6CRQOK3YTAJ3BY6AV4STPY2M, 01m52s elapsed]
aws_db_instance.taller: Destruction complete after 1m55s
aws_security_group.taller: Destroying... [id=sg-0847b8bf416762c4c]
aws_db_subnet_group.taller: Destroying... [id=taller-punto2-db-subnet-group]
aws_db_subnet_group.taller: Destruction complete after 0s
aws_subnet.subnet_2: Destroying... [id=subnet-08398ef2f2e524e64]
aws_subnet.subnet_1: Destroying... [id=subnet-0a000af8ae7795836]
aws_subnet.subnet_1: Destruction complete after 1s
aws_subnet.subnet_2: Destruction complete after 1s
aws_security_group.taller: Destruction complete after 1s
aws_vpc.taller: Destroying... [id=vpc-00c9eddc7c4d6fe92]
aws_vpc.taller: Destruction complete after 1s

Destroy complete! Resources: 8 destroyed.
(aws-taller) eia2024@PC:~/Taller-AWS/punto2_terraform_sdk_cli/terraform$
```

Despliegue con Boto3

- Evidencia 1 – Despliegue completo con Boto3

```
(aws-taller) eia2024@PC:~/Taller-AWS/punto2_terraform_sdk_cli/terraform/punto2_terraform_sdk_cli/boto3$ pyth
on deploy.py
Creando VPC...
VPC creada: vpc-0997e550f7235c967
Subnet 1: subnet-067ce5578ebf051a8
Subnet 2: subnet-030c0e8006dace48f
Security Group creada: sg-02dde7f64237fe70a
Creando bucket taller-aws-p2-211107604485-boto3...
Bucket creado: taller-aws-p2-211107604485-boto3
AMI seleccionada: ami-04b3f3bf99ec556c1
Creando EC2...
EC2 creada: i-0786ee517727a63b5
DB Subnet Group creado
Creando RDS... Esto puede tardar varios minutos.
RDS creada correctamente

--- RESUMEN DEL DESPLIEGUE ---
VPC: vpc-0997e550f7235c967
Subnet 1: subnet-067ce5578ebf051a8
Subnet 2: subnet-030c0e8006dace48f
Security Group: sg-02dde7f64237fe70a
S3: taller-aws-p2-211107604485-boto3
EC2: i-0786ee517727a63b5
RDS: taller-punto2-boto3-rds
Secret ARN: arn:aws:secretsmanager:us-east-2:211107604485:secret:rds!db-3c53dc34-2ae9-4915-b15b-211a3c3667cd
-rWSXcX

Despliegue Boto3 completado.
(aws-taller) eia2024@PC:~/Taller-AWS/punto2_terraform_sdk_cli/terraform/punto2_terraform_sdk_cli/boto3$
```

- Evidencia 2 – Bucket S3 creado con Boto3

```
(aws-taller) eia2024@PC:~/Taller-AWS/punto2_terraform_sdk_cli/terraform/punto2_terraform_sdk_cli/boto3$ aws
s3 ls --profile user_cli
2026-09-22 06:16:28 sgf-demo-bucket
2026-09-29 10:20:16 taller-aws-p2-211107604485-boto3
```

- Evidencia 3 – Instancia EC2 creada con Boto3

```
(aws-taller) eia2024@PC:~/Taller-AWS/punto2_terraform_sdk_cli/terraform/punto2_terraform_sdk_cli/boto3$ aws
ec2 describe-instances \
--region us-east-2 \
--profile user_cli \
--query "Reservations[].Instances[].{ID:InstanceId,State:State.Name,Type:InstanceType}"
[
  {
    "ID": "i-0ff92537236809fc7",
    "State": "terminated",
    "Type": "t3.micro"
  },
  {
    "ID": "i-0786ee517727a63b5",
    "State": "running",
    "Type": "t3.micro"
  }
]
```

- Evidencia 4 – Instancia RDS creada con Boto3

```
(aws-taller) eia2024@PC:~/Taller-AWS/punto2_terraform_sdk_cli/terraform/punto2_terraform_sdk_cli/boto3$ aws
rds describe-db-instances \
--region us-east-2 \
--profile user_cli \
--query "DBInstances[].{ID:DBInstanceIdentifier,Status:DBInstanceStatus,Class:DBInstanceClass,Engine:Engine}"
[
  {
    "ID": "taller-punto2-boto3-rds",
    "Status": "available",
    "Class": "db.t3.micro",
    "Engine": "mysql"
  }
]
```

- Evidencia 5 – Contraseña de RDS administrada por Secrets Manager

```
(aws-taller) eia2024@PC:~/Taller-AWS/punto2_terraform_sdk_cli/terraform/punto2_terraform_sdk_cli/boto3$ aws
rds describe-db-instances \
--db-instance-identifier taller-punto2-boto3-rds \
--region us-east-2 \
--profile user_cli \
--query "DBInstances[0].MasterUserSecret"
{
  "SecretArn": "arn:aws:secretsmanager:us-east-2:211107604485:secret:rds!db-3c53dc34-2ae9-4915-b15b-211a3c3667cd-rWSXcX",
  "SecretStatus": "active",
  "KmsKeyId": "arn:aws:kms:us-east-2:211107604485:key/91775893-f4b9-4121-80a1-17e632ba8549"
}
```

- Evidencia 6 – Eliminación de recursos con Boto3

```
(aws-taller) eia2024@PC:~/Taller-AWS/punto2_terraform_sdk_cli/terraform/punto2_terraform_sdk_cli/boto3$ pyth
on destroy.py
Eliminando RDS...
RDS eliminada correctamente
DB Subnet Group eliminado
Terminando EC2: ['i-0786ee517727a63b5']
EC2 eliminada correctamente
Security Group eliminado: sg-02dde7f64237fe70a
Subnet eliminada: subnet-030c0e8006dace48f
Subnet eliminada: subnet-067ce5578ebf051a8
VPC eliminada: vpc-0997e550f7235c967
Eliminando bucket S3...
Bucket eliminado: taller-aws-p2-211107604485-boto3

--- ELIMINACIÓN COMPLETADA ---
Recursos del despliegue Boto3 eliminados correctamente.
(aws-taller) eia2024@PC:~/Taller-AWS/punto2_terraform_sdk_cli/terraform/punto2_terraform_sdk_cli/boto3$
```

Despliegue con AWS CLI

- Evidencia 1 – VPC y subredes creadas con AWS CLI

```
(aws-taller) eia2024@PC:~/Taller-AWS/punto2_terraform_sdk_cli/terraform/punto2_terraform_sdk_cli/boto3/punto2_terraform_sdk_cli/cli$ echo $VPC_ID
echo $SUBNET1_ID
echo $SUBNET2_ID
vpc-014176f93f06fdcde
subnet-01c027b09d23a6b42
subnet-0427b87b262e33f69
(aws-taller) eia2024@PC:~/Taller-AWS/punto2_terraform_sdk_cli/terraform/punto2_terraform_sdk_cli/boto3/punto2_terraform_sdk_cli/cli$
```

- Evidencia 2 – Security Group creado con AWS CLI

```
(aws-taller) eia2024@PC:~/Taller-AWS/punto2_terraform_sdk_cli/terraform/punto2_terraform_sdk_cli/boto3/punto2_terraform_sdk_cli/cli$ echo $SG_ID
sg-08eae2718cbd40b73
(aws-taller) eia2024@PC:~/Taller-AWS/punto2_terraform_sdk_cli/terraform/punto2_terraform_sdk_cli/boto3/punto2_terraform_sdk_cli/cli$
```

- Evidencia 3 – Bucket S3 creado con AWS CLI

```
(aws-taller) eia2024@PC:~/Taller-AWS/punto2_terraform_sdk_cli/terraform/punto2_terraform_sdk_cli/boto3/punto2_terraform_sdk_cli/cli$ aws s3 ls --profile user_cli
2026-09-22 06:16:28 sgf-demo-bucket
2026-09-29 10:46:03 taller-aws-p2-211107604485-cli
(aws-taller) eia2024@PC:~/Taller-AWS/punto2_terraform_sdk_cli/terraform/punto2_terraform_sdk_cli/boto3/punto2_terraform_sdk_cli/cli$
```

- Evidencia 4 – Instancia EC2 creada con AWS CLI

```
(aws-taller) eia2024@PC:~/Taller-AWS/punto2_terraform_sdk_cli/terraform/punto2_terraform_sdk_cli/boto3/punto2_terraform_sdk_cli/cli$ aws ec2 describe-instances \
--instance-ids $INSTANCE_ID \
--region us-east-2 \
--profile user_cli \
--query "Reservations[0].Instances[0].{ID:InstanceId,State:State.Name,Type:InstanceType}"
{
  "ID": "i-0ec7624ffd757db92",
  "State": "running",
  "Type": "t3.micro"
}
(aws-taller) eia2024@PC:~/Taller-AWS/punto2_terraform_sdk_cli/terraform/punto2_terraform_sdk_cli/boto3/punto2_terraform_sdk_cli/cli$
```

- Evidencia 5 – RDS creada con AWS CLI

```
(aws-taller) eia2024@PC:~/Taller-AWS/punto2_terraform_sdk_cli/terraform/punto2_terraform_sdk_cli/boto3/punto2_terraform_sdk_cli/cli$ aws rds describe-db-instances \
--db-instance-identifier $DB_IDENTIFIER \
--region us-east-2 \
--profile user_cli \
--query "DBInstances[0].{ID:DBInstanceIdentifier,Status:DBInstanceStatus,Class:DBInstanceClass,Engine:Engine}"
{
  "ID": "taller-punto2-cli-rds",
  "Status": "available",
  "Class": "db.t3.micro",
  "Engine": "mysql"
}
(aws-taller) eia2024@PC:~/Taller-AWS/punto2_terraform_sdk_cli/terraform/punto2_terraform_sdk_cli/boto3/punto2_terraform_sdk_cli/cli$
```

- Evidencia 6 – Contraseña de RDS administrada por Secrets Manager

```
(aws-taller) eia2024@PC:~/Taller-AWS/punto2_terraform_sdk_cli/terraform/punto2_terraform_sdk_cli/boto3/punto2_terraform_sdk_cli/cli$ aws rds describe-db-instances \
--db-instance-identifier $DB_IDENTIFIER \
--region us-east-2 \
--profile user_cli \
--query "DBInstances[0].MasterUserSecret"
{
  "SecretArn": "arn:aws:secretsmanager:us-east-2:211107604485:secret:rds!db-f714974c-3483-47fc-9814-58b24587381d-WqiuN5",
  "SecretStatus": "active",
  "KmsKeyId": "arn:aws:kms:us-east-2:211107604485:key/91775893-f4b9-4121-80a1-17e632ba8549"
}
(aws-taller) eia2024@PC:~/Taller-AWS/punto2_terraform_sdk_cli/terraform/punto2_terraform_sdk_cli/boto3/punto2_terraform_sdk_cli/cli$
```

- Evidencia 7 – Resumen de todos los recursos creados

```
(aws-taller) eia2024@PC:~/Taller-AWS/punto2_terraform_sdk_cli/terraform/punto2_terraform_sdk_cli/boto3/punto2_terraform_sdk_cli/cli$ echo "VPC: $VPC_ID"
echo "Subnet 1: $SUBNET1_ID"
echo "Subnet 2: $SUBNET2_ID"
echo "Security Group: $SG_ID"
echo "S3: $BUCKET_NAME"
echo "EC2: $INSTANCE_ID"
echo "RDS: $DB_IDENTIFIER"
VPC: vpc-014176f93f06fcdcde
Subnet 1: subnet-01c027b09d23a6b42
Subnet 2: subnet-0427b87b262e33f69
Security Group: sg-08eae2718cbd40b73
S3: taller-aws-p2-211107604485-cli
EC2: i-0ec7624ffd757db92
RDS: taller-punto2-cli-rds
(aws-taller) eia2024@PC:~/Taller-AWS/punto2_terraform_sdk_cli/terraform/punto2_terraform_sdk_cli/boto3/punto2_terraform_sdk_cli/cli$
```

- Evidencia 8 – Eliminación de recursos

```
remove_bucket: taller-aws-p2-211107604485-cli
(aws-taller) eia2024@PC:~/Taller-AWS/punto2_terraform_sdk_cli/terraform/punto2_terraform_sdk_cli/boto3/punto2_terraform_sdk_cli/cli$ echo "Verificación final"

aws ec2 describe-instances \
--region us-east-2 \
--profile user_cli \
--filters "Name=instance-state-name,Values=running" \
--query "Reservations[].Instances[].InstanceId"

aws rds describe-db-instances \
--region us-east-2 \
--profile user_cli \
--query "DBInstances[].DBInstanceIdentifier"

aws s3 ls --profile user_cli
Verificación final
[]
[]
2026-09-22 06:16:28 sgf-demo-bucket
(aws-taller) eia2024@PC:~/Taller-AWS/punto2_terraform_sdk_cli/terraform/punto2_terraform_sdk_cli/boto3/punto2_terraform_sdk_cli/cli$
```

Comparación de los 3 métodos:

Método	Ventajas	Desventajas	Uso más adecuado
Terraform	Permite definir toda la infraestructura como código, es reproducible y facilita la creación y	Requiere manejar el archivo de estado y aprender HCL.	Infraestructura repetible, ambientes completos y automatización de despliegues.

	eliminación completa con apply y destroy.		
Boto3	Ofrece mucha flexibilidad desde Python y permite integrar la infraestructura con lógica de programación.	Requiere más código y manejar manualmente el orden de creación y eliminación de recursos.	Automatizaciones personalizadas y aplicaciones que necesitan interactuar con AWS.
AWS CLI	Es rápido, directo y útil para ejecutar operaciones puntuales desde terminal.	Se vuelve más difícil de mantener cuando hay muchos recursos y dependencias.	Pruebas, administración rápida y tareas individuales.

Explique cómo autentica user_cli las herramientas.

R/ El usuario IAM user_cli se autenticó mediante un perfil local configurado con AWS CLI usando `aws configure --profile user_cli`. En este perfil se almacenaron el Access Key ID, el Secret Access Key, la región y el formato de salida. Posteriormente, AWS CLI utilizó este perfil mediante la opción `--profile user_cli`, Boto3 lo utilizó mediante `boto3.Session(profile_name="user_cli")` y Terraform lo utilizó configurando `profile = "user_cli"` dentro del proveedor de AWS. De esta manera, las tres herramientas trabajaron con el mismo usuario IAM y no fue necesario utilizar credenciales de la cuenta raíz.

Revise también qué información conserva Terraform en su archivo de estado y evite introducir allí contraseñas.

R/ Terraform conserva información de los recursos que administra dentro del archivo `terraform.tfstate`. Este archivo puede incluir identificadores de recursos, ARN, nombres, configuraciones, relaciones entre recursos y, dependiendo de cómo se definan, también valores sensibles.

¿Cuánto costaría mantener los recursos durante una hora y durante una semana?

R/ El costo estimado de mantener la infraestructura desplegada durante una hora es de aproximadamente USD 0,03, mientras que mantenerla durante una semana completa (168 horas) costaría aproximadamente USD 5,4–5,6.

¿Qué componentes seguirían generando cargos sin recibir tráfico?

R/ Los recursos que pueden continuar generando cargos aunque no reciban tráfico son principalmente la instancia EC2 mientras permanezca en ejecución, la instancia RDS, el

almacenamiento de RDS, el volumen EBS asociado a EC2, los objetos almacenados en S3 y el secreto de Secrets Manager.

¿Qué costo añade el secreto de RDS?

R/ El secreto de RDS administrado por AWS Secrets Manager añade aproximadamente USD 0,40 por secreto al mes, prorrateado por el tiempo que permanezca almacenado, más USD 0,05 por cada 10.000 llamadas API.

3. Amazon DocumentDB

Compare DocumentDB con MongoDB e identifique al menos tres diferencias que podrían afectar una aplicación existente.

R/ Amazon DocumentDB es un servicio administrado de AWS compatible con las APIs de MongoDB, pero no es MongoDB y existen diferencias que pueden afectar una aplicación existente. Tres diferencias importantes son:

- **Compatibilidad parcial de comandos y operadores.** DocumentDB soporta gran parte de la API de MongoDB, pero algunos comandos, operadores y comportamientos no están disponibles o funcionan de forma diferente.
- **Planes de ejecución y rendimiento.** DocumentDB utiliza un motor propio de AWS, por lo que los planes de consulta y la salida de explain() pueden ser diferentes a MongoDB. Esto puede cambiar el rendimiento de consultas existentes.
- **Diferencias en índices y comportamiento de consultas.** Algunas características de indexación, ordenamiento, \$lookup, índices sparse, búsquedas y otros operadores tienen diferencias funcionales respecto a MongoDB.

Despliegue DocumentDB con una configuración de bajo costo. Compare una instancia aprovisionada con DocumentDB Serverless, disponible con una capacidad mínima configurable de 0,5 DCU, y justifique cuál resulta más económica para su región y tiempo de uso.

R/ Trabajamos en la región us-east-2 (Ohio), la misma de todo el taller, y comparamos las configuraciones más pequeñas que ofrece DocumentDB. Para el tiempo de uso tomamos unas tres horas, que fue lo que nos tomó desplegar, conectar, ejecutar las consultas y eliminar los recursos.

Opción	Capacidad mínima	Precio por hora	Costo estimado (3 h)
Instancia aprovisionada db.t4g.medium	2 vCPU y 4 GiB de RAM, fija	USD 0,076	≈ USD 0,23
Instancia aprovisionada db.t3.medium	2 vCPU y 4 GiB de RAM, fija	USD 0,078	≈ USD 0,23
DocumentDB Serverless (Standard)	0,5 DCU (≈ 1 GiB de RAM)	$0,5 \times \text{USD } 0,0822 = \text{USD } 0,0411$	≈ USD 0,12

Elegimos DocumentDB Serverless con una capacidad entre 0,5 y 1 DCU. Nuestra carga era mínima: diez documentos y tres consultas. Con una instancia aprovisionada habríamos pagado una máquina de 4 GiB todo el tiempo que estuviera encendida, aunque casi no la usáramos. Serverless, en cambio, se cobra por segundo según la capacidad que realmente consume, y su mínimo de 0,5 DCU cuesta casi la mitad por hora que la instancia aprovisionada más barata. Además, al fijar el máximo en 1 DCU nos aseguramos de que el costo no se disparara aunque hubiera un pico de carga.

El clúster lo creamos desde AWS CloudShell con el usuario `juan_admin`. Primero verificamos con `describe-orderable-db-instance-options` que la versión 5.0.0 del motor admitía instancias `db.serverless` en Ohio. Luego creamos un clúster llamado `docdb-taller` con una instancia `docdb-taller-1` de tipo `db.serverless`, retención de backups de un día y sin protección contra eliminación, para poder borrarlo fácilmente al final.

Algo que aprendimos al comparar es que Serverless no siempre es la opción más barata. No se apaga del todo, porque siempre se cobra el mínimo de 0,5 DCU, y si una aplicación mantiene una carga alta y constante, una instancia aprovisionada termina saliendo más económica. Serverless conviene para cargas pequeñas, intermitentes o de pruebas como la nuestra. También tuvimos en cuenta que AWS ofrece una prueba gratuita de un mes de DocumentDB para quienes usan el servicio por primera vez, pero ese beneficio aplica solo a instancias `db.t3.medium` y no a Serverless.

¿Por qué DocumentDB no se puede consultar directamente desde el computador?

R/ DocumentDB se despliega dentro de una VPC y no tiene una dirección IP pública. Solo acepta conexiones de recursos que estén dentro de esa misma red privada y que los grupos de seguridad autoricen. Para conectarnos desde nuestro computador tuvimos que usar un intermediario: una instancia EC2 ubicada en la misma VPC, a la que sí podemos llegar por internet mediante SSH, y que reenvía el tráfico hacia el clúster.

Prepare una instancia EC2 en la misma VPC y un túnel SSH. Configure los grupos de seguridad, TLS y el certificado necesarios.

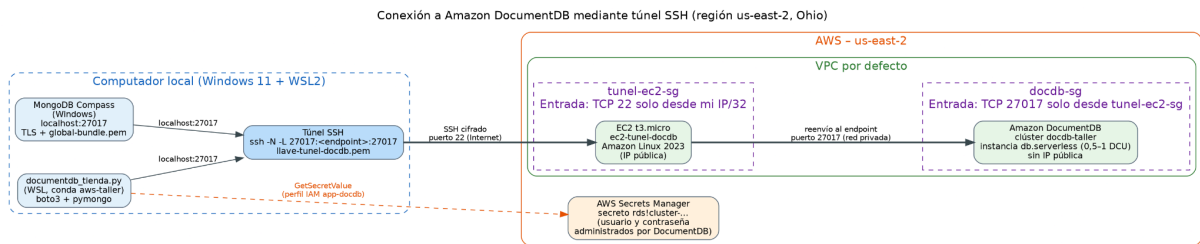
R/ Para la conexión configuramos lo siguiente:

- **Red:** usamos la VPC por defecto de us-east-2. Creamos el grupo de subredes docdb-taller-subnets con tres subredes en las zonas us-east-2a, us-east-2b y us-east-2c, y ubicamos tanto el clúster de DocumentDB como la EC2 dentro de esa misma VPC.
- **EC2 del túnel:** una instancia t3.micro con Amazon Linux 2023, IP pública y un par de llaves RSA (llave-tunnel-docdb.pem), guardado en WSL con permisos 400 para que solo nuestro usuario pudiera leerlo. No instalamos nada en ella, porque solo funciona como puente.
- **Grupos de seguridad:** creamos dos. El grupo tunnel-ec2-sg permite SSH (puerto 22) únicamente desde nuestra IP pública con máscara /32. El grupo docdb-sg permite el puerto 27017 únicamente cuando el origen es el grupo tunnel-ec2-sg. Así, aunque alguien conozca el endpoint del clúster, no puede conectarse si no está dentro de nuestra EC2. Al principio intentamos nombrar los grupos con el prefijo sg-, pero AWS no lo permite porque lo reserva para los identificadores de los grupos.
- **Túnel SSH:** desde WSL ejecutamos `ssh -i llave-tunnel-docdb.pem -N -L 27017:<endpoint del clúster>:27017 ec2-user@<DNS de la EC2>`. Con esto, todo lo que enviamos a localhost:27017 viaja cifrado hasta la EC2 y desde ahí llega al clúster. La primera vez SSH nos pidió confirmar la huella de la EC2, que queda guardada para las siguientes conexiones. Como WSL2 comparte localhost con Windows, el mismo túnel nos sirvió para MongoDB Compass y para Python.
- **TLS y certificado:** DocumentDB tiene TLS activado por defecto, así que descargamos el certificado global-bundle.pem de AWS y lo usamos tanto en Compass como en Python para validar la conexión cifrada. Activamos `tlsAllowInvalidHostnames` porque el certificado está emitido para el endpoint del clúster y nosotros nos conectábamos a localhost: los nombres no coinciden, pero la conexión sigue cifrada. También usamos `directConnection=true`, porque nos conectamos a un solo nodo a través del túnel, y `retryWrites=false`, porque DocumentDB no soporta escrituras reintentables.

Compruebe la conexión desde MongoDB Compass y desde Python.

R/ En MongoDB Compass creamos una conexión a localhost:27017 con conexión directa, autenticación por usuario y contraseña (tomados del secreto en Secrets Manager), TLS activado con el certificado global-bundle.pem, la opción `tlsAllowInvalidHostnames` y `retryWrites=false`. Al principio la lista de bases de datos apareció vacía, lo cual era normal porque el clúster estaba recién creado. Para confirmar que realmente estábamos conectados a DocumentDB y no a otro servidor, abrimos la shell integrada de Compass y ejecutamos `db.version()`, que respondió 5.0.0, la misma versión del motor del clúster. Desde Python la conexión se comprobó con un ping al servidor, que respondió `{'ok': 1.0}`.

Diagrama de conexión (lo que se hizo para poder conectar DocumentDB a Compass)



Gestione la contraseña de DocumentDB con Secrets Manager y recupérela desde el programa.

R/ Al crear el clúster usamos la opción de credenciales administradas (--manage-master-user-password). Con ella, nunca escribimos la contraseña del usuario administrador docdbadmin: AWS la generó automáticamente y la guardó en un secreto llamado rds!cluster-... dentro de Secrets Manager, cifrado con una llave KMS administrada por AWS. Para que el programa pudiera leerla, creamos el usuario IAM app_docdb con una política que solo permite la acción secretsmanager:GetSecretValue sobre el ARN de ese secreto. Lo comprobamos intentando listar los demás secretos de la cuenta con ese usuario, y AWS respondió con AccessDeniedException. El programa en Python usa Boto3 con el perfil app-docdb para recuperar el usuario y la contraseña en tiempo de ejecución, así que la contraseña no aparece en el código ni en el repositorio. Una ventaja adicional es que AWS puede rotar esa contraseña sin que tengamos que modificar el programa. Cuando eliminamos el clúster, AWS borró automáticamente el secreto asociado.

Cree una base de datos y una colección, inserte al menos diez documentos y ejecute una búsqueda con filtro, una ordenación y una agregación. Todo esto usando Python.

R/ Con el programa documentdb_tienda.py creamos la base de datos tienda y la colección productos, e insertamos diez productos con los campos nombre, categoría, precio, stock y ciudad. Luego ejecutamos tres consultas:

- **Consulta 1 – Búsqueda con filtro:** `find({"categoria": "Tecnología", "precio": {"$gt": 200000}})`. Responde a la pregunta: ¿qué productos de tecnología cuestan más de 200.000? El resultado fueron tres productos: el *Portátil Lenovo* (3.200.000), el *Monitor de 27 pulgadas* (1.100.000) y los *Audífonos Bluetooth* (250.000). El mouse inalámbrico quedó por fuera porque, aunque es de tecnología, cuesta \$85.000.
- **Consulta 2 – Ordenación:** `find({"ciudad": "Medellín"}).sort("precio", -1)`. Responde a la pregunta: ¿cuáles son los productos disponibles en Medellín, del más caro al más barato? Aparecieron cuatro productos en este orden: *Portátil Lenovo* (3.200.000), *Monitor de 27 pulgadas* (1.100.000), *Escritorio en L* (780.000) y *Café de Antioquia* (32.000). El -1 indica orden descendente.
- **Consulta 3 – Agregación:** un pipeline con \$group por categoría, que cuenta los productos, suma el stock y calcula el precio promedio, seguido de un \$sort por stock descendente. Responde a la pregunta: ¿cómo se distribuye el inventario entre categorías? Alimentos tiene el mayor stock (140 unidades en 2 productos, con un

precio promedio de \$20.750), seguido de Tecnología (78 unidades en 4 productos, con el precio promedio más alto: \$1.158.750), Oficina (33 unidades en 3 productos, con un promedio de \$616.667) y Hogar (30 unidades en 1 producto, a \$65.000). Esto muestra que las categorías con más unidades son las de productos baratos, mientras que tecnología concentra el mayor valor por producto con menos inventario.

Después de ejecutar el programa, verificamos desde MongoDB Compass que la base de datos tienda y la colección productos aparecían con los diez documentos insertados.

Preguntas de costos

¿Qué opción de DocumentDB cuesta menos para esta práctica?

R/ Para nuestra práctica la opción más económica fue DocumentDB Serverless con el mínimo de 0,5 DCU, que cuesta unos USD 0,041 por hora. La instancia aprovisionada más barata en Ohio es la db.t4g.medium, a USD 0,076 por hora, seguida de la db.t3.medium, a USD 0,078. En unas tres horas de uso, el cómputo de Serverless quedó en aproximadamente USD 0,12, mientras que cualquiera de las dos instancias aprovisionadas habría costado unos USD 0,23, casi el doble. A esto se suman otros cobros casi despreciables en nuestro caso:

- **Almacenamiento:** USD 0,10 por GB-mes, y nuestros diez documentos ocupan unos pocos KB.
- **Operaciones de entrada y salida:** USD 0,20 por millón, y hicimos apenas unas decenas.
- **El secreto administrado:** USD 0,40 al mes prorrateado, que en tres horas son unos USD 0,002.

La única situación en la que una instancia aprovisionada sería más barata es si la cuenta tuviera activa la prueba gratuita de DocumentDB, que cubre 750 horas de db.t3.medium durante el primer mes. En ese caso su uso no tendría costo.

¿Cuánto agrega la EC2 del túnel?

R/ La EC2 del túnel fue una t3.micro, que cuesta alrededor de USD 0,0104 por hora. A eso se suma el cobro por la dirección IPv4 pública (USD 0,005 por hora) y el disco EBS de 8 GB (unos USD 0,64 al mes, prorrateado).

Concepto	Costo estimado (3 h)
DocumentDB Serverless (0,5 DCU)	≈ USD 0,12
EC2 t3.micro	≈ USD 0,031
IPv4 pública	≈ USD 0,015
Disco EBS 8 GB	≈ USD 0,003

Secreto de Secrets Manager	≈ USD 0,002
Almacenamiento e I/O de DocumentDB	≈ USD 0,00
Total de la práctica	≈ USD 0,17

En nuestro caso la EC2 agregó unos USD 0,05, casi un 30 % del total. Aunque parece poco, nos dimos cuenta de que es fácil olvidarla. Si la dejáramos encendida un mes completo, costaría alrededor de USD 11,9 (730 horas de instancia e IP pública, más el disco) sin recibir ni una consulta. Por eso la terminamos apenas acabamos la práctica.

Eliminación de recursos

R/ Al terminar eliminamos todo desde CloudShell, en este orden:

1. La instancia docdb-taller-1, porque un clúster no se puede borrar mientras tenga instancias.
2. El clúster docdb-taller, sin snapshot final para que no quedara almacenamiento cobrándose.
3. La EC2 del túnel.
4. El grupo de subredes.
5. Los grupos de seguridad: primero docdb-sg, porque tenía una regla que hacía referencia a tunel-ec2-sg, y después tunel-ec2-sg.
6. El par de llaves.

Por último, borramos el usuario IAM app_docdb con su clave de acceso y la llave .pem de nuestro computador. Para comprobar que no quedara nada facturable, verificamos que no existieran clústeres, snapshots, volúmenes EBS sueltos ni el secreto rds!cluster-..., y que la EC2 estuviera en estado terminated.

Entregables:

- **Código Python:** está en GitHub (punto3_documentdb/documentdb_tienda.py).
- **Política de acceso del programa:** está en GitHub (punto3_documentdb/politica_app_docdb.json).
- **Consultas:** filtro, ordenación y agregación, incluidas en el código y explicadas arriba.
- **Diagrama de conexión:** incluido arriba.
- **Cálculo de costos:** incluido en las preguntas de costos.

- Evidencia 1 - Variables de Red

```
CloudShell
us-east-2 +
~ $
~ $ VPC_ID=$(aws ec2 describe-vpcs --filters Name=isDefault,Values=true --query "Vpcs[0].VpcId" --output text)
~ $
~ $ SUBNETS=$(aws ec2 describe-subnets --filters Name=vpc-id,Values=$VPC_ID Name=availability-zone,Values=us-east-2a,us-east-2b,us-east-2c --query "Subnets[].SubnetId" --output text)
~ $
~ $ SUBNET_EC2=$(echo $SUBNETS | awk '{print $1}')
~ $
~ $ MI_IP="186.121.37.4"
~ $
~ $ echo "VPC_ID=$VPC_ID" > ~/punto3.env
~ $ echo "SUBNETS=$SUBNETS" > ~/punto3.env
~ $ echo "SUBNET_EC2=$SUBNET_EC2" > ~/punto3.env
~ $ echo "MI_IP=$MI_IP" > ~/punto3.env
~ $ cat ~/punto3.env
VPC_ID=vpc-05d67dab7b3f5c5      subnet-0a2ceda3e4c579cd      subnet-09f9f1ff1e167c91d"
SUBNETS=subnet-0e0cf9b65163746a7
SUBNET_EC2=subnet-0e0cf9b65163746a7
MI_IP=186.121.37.4
~ $
~ $
```

- Evidencia 2 - Creación de grupos de seguridad (EC2)

```
CloudShell
us-east-2 +
~ $ source ~/punto3.env
~ $
~ $ SG_EC2=$(aws ec2 create-security-group --group-name tunel-ec2-sg \
> --description "SSH para tunel hacia DocumentDB" --vpc-id $VPC_ID \
> --query GroupId --output text)
~ $
~ $ echo "SG_EC2 = $SG_EC2"
SG_EC2 = sg-0bc834f9812e4ebd2
~ $
```

```
~ $ aws ec2 authorize-security-group-ingress --group-id $SG_EC2 \
> --protocol tcp --port 22 --cidr $MI_IP/32
{
  "Return": true,
  "SecurityGroupRules": [
    {
      "SecurityGroupRuleId": "sgr-0e503dc7ab9db48ac",
      "GroupId": "sg-0bc834f9812e4ebd2",
      "GroupOwnerId": "052022403573",
      "IsEgress": false,
      "IpProtocol": "tcp",
      "FromPort": 22,
      "ToPort": 22,
      "CidrIpv4": "186.121.37.4/32",
      "SecurityGroupRuleArn": "arn:aws:ec2:us-east-2:052022403573:security-group-rule/sgr-0e503dc7ab9db48ac"
    }
  ]
}
~ $
~ $ echo "SG_EC2=$SG_EC2" >> ~/punto3.env
```

- Evidencia 3 - Creación de grupos de seguridad (DocumentDB)

```

~ $ SG_DOCDB=$(aws ec2 create-security-group --group-name docdb-sg \
> --description "DocumentDB solo desde la EC2 del tunel" --vpc-id $VPC_ID \
> --query GroupId --output text)
~ $
~ $ echo "SG_DOCDB = $SG_DOCDB"
SG_DOCDB = sg-0b0971a79473b899c
~ $

```

```

~ $ aws ec2 authorize-security-group-ingress --group-id $SG_DOCDB \
> --protocol tcp --port 27017 --source-group $SG_EC2
{
  "Return": true,
  "SecurityGroupRules": [
    {
      "SecurityGroupRuleId": "sgr-0b669b2675a40e8cf",
      "GroupId": "sg-0b0971a79473b899c",
      "GroupOwnerId": "052022403573",
      "IsEgress": false,
      "IpProtocol": "tcp",
      "FromPort": 27017,
      "ToPort": 27017,
      "ReferencedGroupInfo": {
        "GroupId": "sg-0bc834f9812e4ebd2",
        "UserId": "052022403573"
      },
      "SecurityGroupRuleArn": "arn:aws:ec2:us-east-2:052022403573:security-group-rule/sgr-0b669b2675a40e8cf"
    }
  ]
}
~ $

```

```

      "SecurityGroupRuleId": "sgr-0b669b2675a40e8cf",
      "IsEgress": false,
      "IpProtocol": "tcp",
      "FromPort": 27017,
      "ToPort": 27017,
      "ReferencedGroupInfo": {
        "GroupId": "sg-0bc834f9812e4ebd2",
        "UserId": "052022403573"
      },
      "SecurityGroupRuleArn": "arn:aws:ec2:us-east-2:052022403573:security-group-rule/sgr-0b669b2675a40e8cf"
    }
  ]
}
~ $
~ $ echo "SG_DOCDB=$SG_DOCDB" >> ~/punto3.env
~ $ cat ~/punto3.env
VPC_ID=vpc-052de7c8ab7b3f5c5
SUBNETS="subnet-0e8cf9b65163746a7      subnet-0a2ceda3e4cb579c8      subnet-09f9f1ff1e167c92d"
SUBNET_EC2=subnet-0e8cf9b65163746a7
MI_IP=186.121.37.4
SG_EC2=sg-0bc834f9812e4ebd2
SG_DOCDB=sg-0b0971a79473b899c
~ $

```

- Evidencia 4 - Variables de entorno (.env)

```

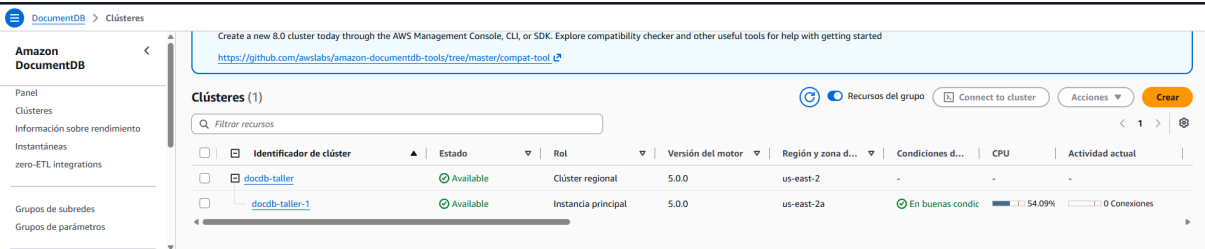
~ $ cat ~/punto3.env
VPC_ID=vpc-052de7c8ab7b3f5c5
SUBNETS="subnet-0e8cf9b65163746a7      subnet-0a2ceda3e4cb579c8      subnet-09f9f1ff1e167c92d"
SUBNET_EC2=subnet-0e8cf9b65163746a7
MI_IP=186.121.37.4
SG_EC2=sg-0bc834f9812e4ebd2
SG_DOCDB=sg-0b0971a79473b899c
~ $

```

- Evidencia 5 - Versiones de DocumentDB compatibles con Serverless en us-east-2

```
~$ aws docdb describe-orderable-db-instance-options --engine docdb \
> --db-instance-class db.serverless \
> --query "OrderableDBInstanceOptions[].EngineVersion" --output text
5.0.0 5.0.0 5.0.1 5.0.1 5.0.2 5.0.2 8.0.0 8.0.0 8.0.1 8.0.1 8.0.2 8.0.2
~$
```

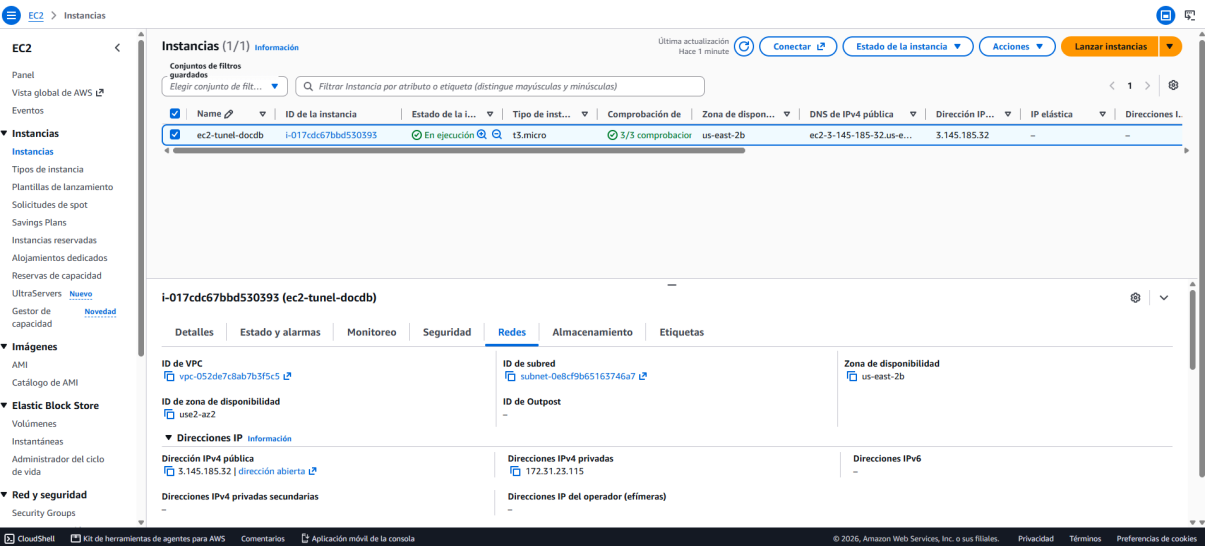
- Evidencia 6 - Clúster DocumentDB Serverless creado



- Evidencia 7 - Contraseña de DocumentDB administrada por Secrets Manager



- Evidencia 8 - Instancia EC2 del túnel en la misma VPC:



- Evidencia 9 - Precios de DocumentDB en us-east-2

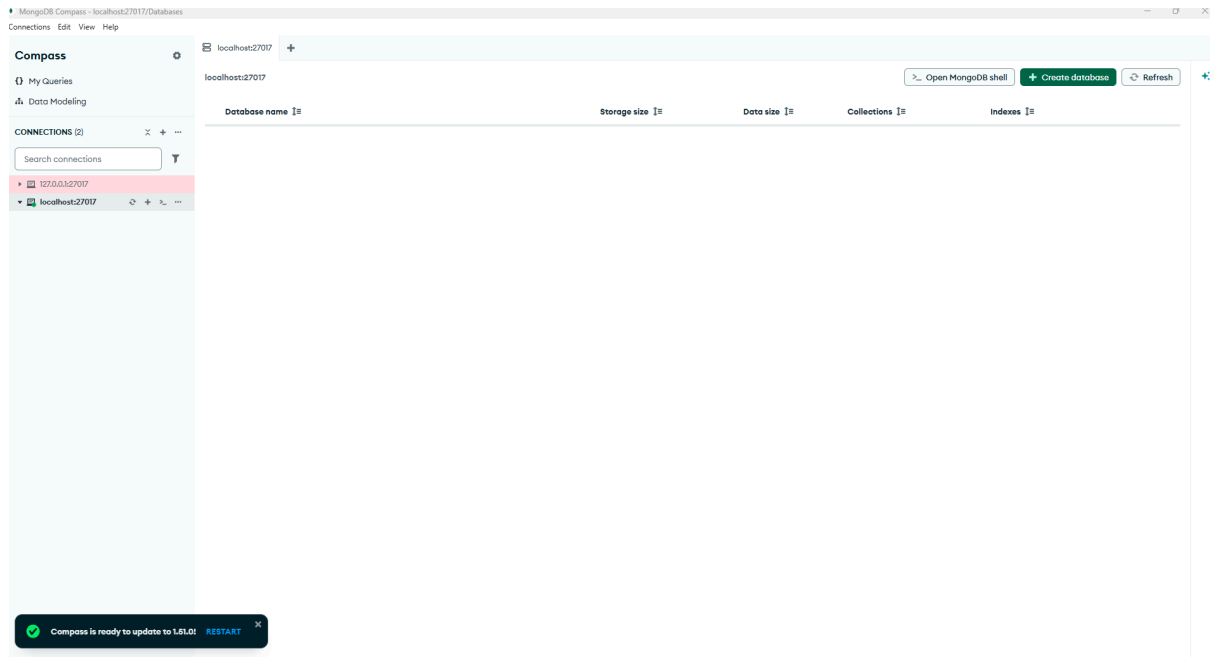
Instancias de uso general: generación actual	Memoria	vCPU	Amazon DocumentDBEstándar (precio por hora)	Amazon DocumentDBOptimizado para E/S (precio por hora)
db.t4g.medium	4 GiB	2	0,076 USD	0,083 USD
db.t3.medium	4 GiB	2	0,078 USD	0,086 USD

- Evidencia 10 - Política de mínimo privilegio para el programa Python

- Evidencia 11 – Túnel SSH activo hacia DocumentDB

```
(base) juanjo@DESKTOP-PGAPLBA:~/Taller-AWS$ conda activate aws-taller
(aws-taller) juanjo@DESKTOP-PGAPLBA:~/Taller-AWS$ nc -zv localhost 27017
Connection to localhost (127.0.0.1) 27017 port [tcp/*] succeeded!
(aws-taller) juanjo@DESKTOP-PGAPLBA:~/Taller-AWS$
```

- Evidencia 12 – Conexión desde MongoDB Compass



- Evidencia 13 – Ejecución del programa Python (inserción, filtro, ordenación y agregación)

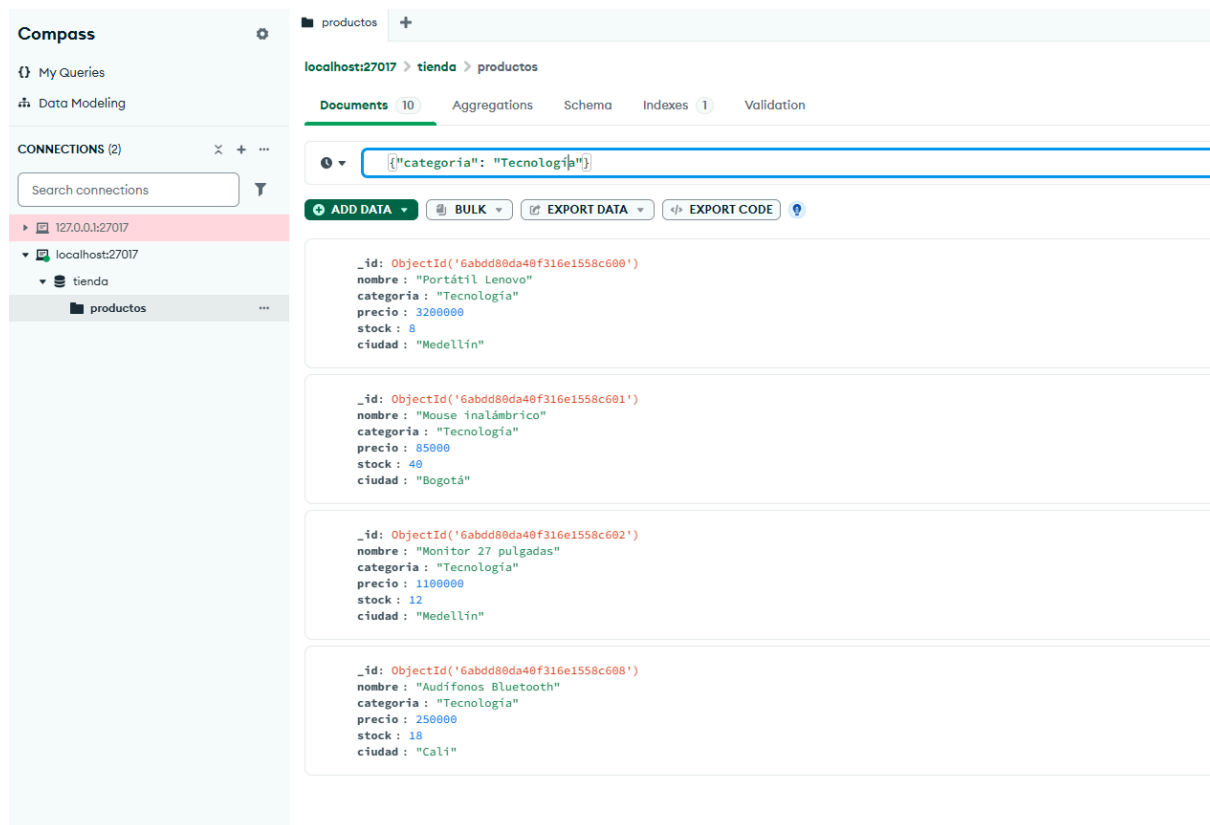
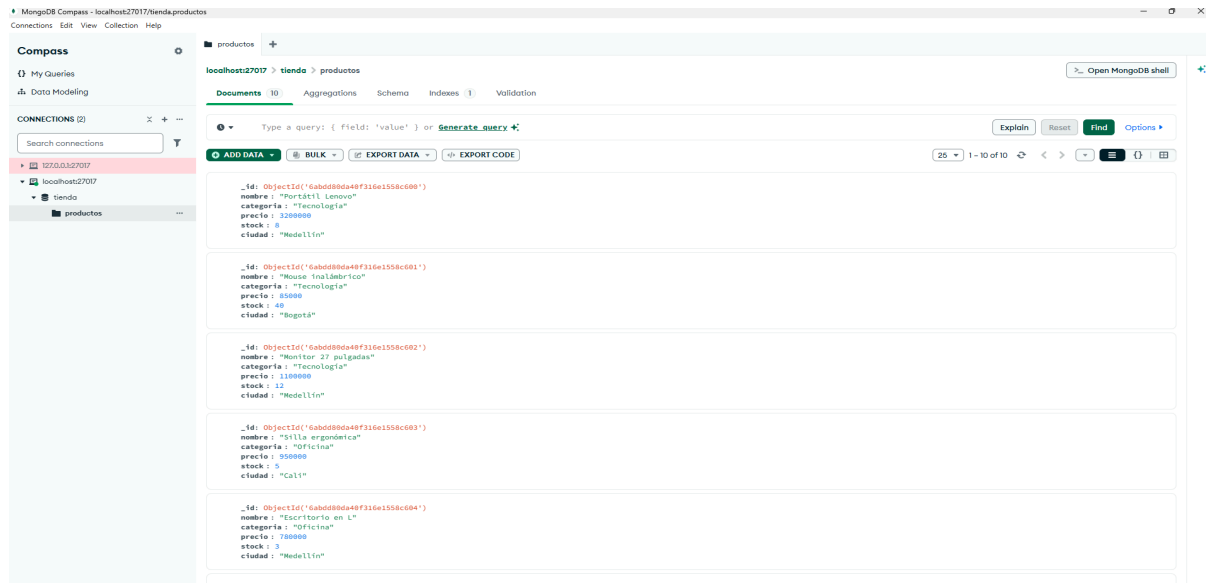
```
Problemas Salida Terminal Puertos Consola de depuración
(aws-taller) juanjo@DESKTOP-PGAPLBA:~/Taller-AWS$
(aws-taller) juanjo@DESKTOP-PGAPLBA:~/Taller-AWS$
(aws-taller) juanjo@DESKTOP-PGAPLBA:~/Taller-AWS$ cd ~/Taller-AWS/punto3_documentdb
(aws-taller) juanjo@DESKTOP-PGAPLBA:~/Taller-AWS/punto3_documentdb$ python documentdb_tienda.py
Ping: {'ok': 1.0, 'operationTime': Timestamp(1790826510, 1)}
Documentos insertados: 10

Consulta 1 - Tecnología con precio > 200.000:
{'nombre': 'Portátil Lenovo', 'precio': 3200000}
{'nombre': 'Monitor 27 pulgadas', 'precio': 1100000}
{'nombre': 'Audífonos Bluetooth', 'precio': 250000}

Consulta 2 - Productos de Medellín ordenados por precio descendente:
{'nombre': 'Portátil Lenovo', 'precio': 3200000}
{'nombre': 'Monitor 27 pulgadas', 'precio': 1100000}
{'nombre': 'Escritorio en L', 'precio': 780000}
{'nombre': 'Café de Antioquia 500 g', 'precio': 32000}

Consulta 3 - Resumen por categoría:
Alimentos | productos: 2 | stock: 140 | precio promedio: 20750
Tecnología | productos: 4 | stock: 78 | precio promedio: 1158750
Oficina | productos: 3 | stock: 33 | precio promedio: 616667
Hogar | productos: 1 | stock: 30 | precio promedio: 65000
(aws-taller) juanjo@DESKTOP-PGAPLBA:~/Taller-AWS/punto3_documentdb$
```

- Evidencia 14 – Documentos insertados vistos desde MongoDB Compass



- Evidencia 15 – Eliminación de DocumentDB, EC2 y red

```
us-east-2 X us-east-2 X +
name : running
}
}
~ $
~ $
~ $ aws ec2 terminate-instances --instance-ids $EC2_ID
{
  "TerminatingInstances": [
    {
      "InstanceId": "i-017cdc67bbd530393",
      "CurrentState": {
        "Code": 32,
        "Name": "shutting-down"
      },
      "PreviousState": {
        "Code": 32,
        "Name": "shutting-down"
      }
    }
  ]
}
~ $
~ $ aws ec2 wait instance-terminated --instance-ids $EC2_ID
~ $
~ $ while aws docdb describe-db-clusters --db-cluster-identifier docdb-taller >>/dev/null 2>&1; do echo "Borrando clúster..."; sleep 30; done; echo "Clúster eliminado"
Clúster eliminado
~ $
~ $
~ $ aws docdb delete-db-subnet-group --db-subnet-group-name docdb-taller-subnets
~ $
~ $
~ $ aws ec2 delete-security-group --group-id $SG_DOCDB
{
  "Return": true,
  "GroupId": "sg-0b9971a79473b899c"
}
~ $
~ $
~ $ aws ec2 delete-security-group --group-id $SG_EC2
{
  "Return": true,
  "GroupId": "sg-0bc834f9812e4ebd2"
}
~ $
~ $ aws ec2 delete-key-pair --key-name llave-tunnel-docdb
{
  "Return": true,
  "KeyPairId": "key-081e7c07311389a95"
}
~ $
~ $
```

4. Amazon Neptune

Compare Neptune con Neo4j en modelos de datos, lenguajes de consulta y herramientas.

R/ Amazon Neptune y Neo4j son bases de datos orientadas a grafos, pero tienen diferencias importantes.

Amazon Neptune es un servicio administrado de AWS orientado a aplicaciones de grafos a gran escala. Soporta principalmente dos modelos: Property Graph y RDF. Para consultas utiliza Gremlin, openCypher y SPARQL, según el modelo de datos utilizado.

Neo4j utiliza principalmente el modelo Property Graph y su lenguaje de consulta más característico es Cypher. También ofrece herramientas propias como Neo4j Browser, Neo4j Bloom y Neo4j Desktop, orientadas a explorar y visualizar grafos.

En cuanto a herramientas, Neptune se integra fuertemente con el ecosistema AWS, incluyendo IAM, VPC, CloudWatch y otros servicios. Neo4j, en cambio, dispone de un ecosistema propio más enfocado en desarrollo, análisis visual y exploración interactiva de grafos.

Despliegue Neptune con una configuración de bajo costo y justifique la elección entre capacidad aprovisionada y Serverless, si ambas están disponibles.

R/ Para el taller conviene usar Amazon Neptune Serverless, porque la carga será pequeña, temporal y variable. Neptune Serverless ajusta automáticamente la capacidad de cómputo según la demanda. La alternativa aprovisionada asigna una instancia fija y cobra por esa capacidad mientras permanezca activa. Serverless, en cambio, cobra según las NCU utilizadas y puede reducirse hasta el mínimo configurado cuando la carga baja. AWS recomienda comparar ambos modelos según el uso esperado, pero para cargas intermitentes o de laboratorio Serverless puede resultar más conveniente.

Investigue cómo consultar Neptune desde local.

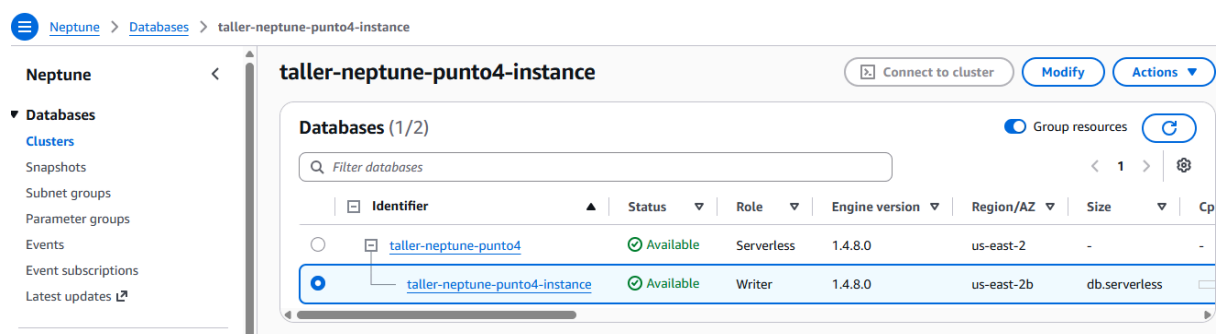
R/ Amazon Neptune normalmente se ejecuta dentro de una VPC, por lo que un computador local no siempre puede conectarse directamente. Una forma común es usar una instancia EC2 dentro de la misma VPC como intermediaria y crear un túnel SSH desde el equipo local hacia esa EC2. A través de ese túnel, las consultas se redirigen al endpoint de Neptune por el puerto 8182.

Compare un túnel SSH mediante EC2 con un punto de acceso público de Neptune, si la versión utilizada lo permite.

R/ Un túnel SSH mediante EC2 permite acceder a Neptune manteniéndolo privado dentro de la VPC. El computador local se conecta primero a una instancia EC2 y, a través de ella, se redirige el tráfico hacia Neptune. Esta opción ofrece mayor aislamiento de red, pero requiere administrar una EC2 adicional y puede generar costo mientras la instancia esté encendida.

Si la versión de Neptune utilizada permite un punto de acceso público, se puede conectar directamente desde el computador local sin usar una EC2 intermedia. Esto simplifica la arquitectura y evita el costo de la instancia auxiliar, pero exige una configuración más cuidadosa de seguridad, especialmente mediante IAM y grupos de seguridad restringidos.

- Evidencia 1 – Clúster Amazon Neptune Serverless creado y disponible



- Evidencia 2 – Configuración de acceso público y Security Group en puerto 8182

Connectivity & Security	Monitoring	Logs & Events	Configuration	Maintenance	Tags															
Connectivity & security <table border="1"> <thead> <tr> <th>Endpoint & port</th> <th>Networking</th> <th>Security</th> </tr> </thead> <tbody> <tr> <td> Endpoint taller-neptune-punto4-instance.c34ug8q4agsj.us-east-2.neptune.amazonaws.com </td> <td> Availability zone us-east-2b </td> <td> VPC security groups neptune-punto4-sg (sg-0dd97d2365550e713) ↗ ✔ Active </td> </tr> <tr> <td> Port 8182 </td> <td> VPC vpc-066a250055233310a ↗ </td> <td> Public connectivity Yes </td> </tr> <tr> <td></td> <td> Subnet group default-vpc-066a250055233310a </td> <td></td> </tr> <tr> <td></td> <td> Subnets subnet-0cb120d9af1763c33 ↗ ✔ Active subnet-012a52d6814c15a22 ↗ ✔ Active subnet-0264d2d83cd892d79 ↗ ✔ Active </td> <td></td> </tr> </tbody> </table>						Endpoint & port	Networking	Security	Endpoint taller-neptune-punto4-instance.c34ug8q4agsj.us-east-2.neptune.amazonaws.com	Availability zone us-east-2b	VPC security groups neptune-punto4-sg (sg-0dd97d2365550e713) ↗ ✔ Active	Port 8182	VPC vpc-066a250055233310a ↗	Public connectivity Yes		Subnet group default-vpc-066a250055233310a			Subnets subnet-0cb120d9af1763c33 ↗ ✔ Active subnet-012a52d6814c15a22 ↗ ✔ Active subnet-0264d2d83cd892d79 ↗ ✔ Active	
Endpoint & port	Networking	Security																		
Endpoint taller-neptune-punto4-instance.c34ug8q4agsj.us-east-2.neptune.amazonaws.com	Availability zone us-east-2b	VPC security groups neptune-punto4-sg (sg-0dd97d2365550e713) ↗ ✔ Active																		
Port 8182	VPC vpc-066a250055233310a ↗	Public connectivity Yes																		
	Subnet group default-vpc-066a250055233310a																			
	Subnets subnet-0cb120d9af1763c33 ↗ ✔ Active subnet-012a52d6814c15a22 ↗ ✔ Active subnet-0264d2d83cd892d79 ↗ ✔ Active																			

- Evidencia 3 – Conexión exitosa desde el equipo local a Neptune

```
(aws-taller) eia2024@PC: ~/Taller-AWS/punto4_neptune$ aws neptunedata execute-open-cypher-query \
--profile user_cli \
--region us-east-2 \
--endpoint-url https://taller-neptune-punto4-instance.c34ug8q4agsj.us-east-2.neptune.amazonaws.com:8182 \
--open-cypher-query "MATCH (n) RETURN n LIMIT 5"
{
  "results": []
}
```

- Evidencia 4 – Ejecución del script Python y creación del grafo

```
(aws-taller) eia2024@PC: ~/Taller-AWS/punto4_neptune$ python neptune_grafo.py
Creando vertices...
10 vertices creados correctamente.
Creando relaciones...
18 relaciones creadas correctamente.
```

- Evidencia 5 – Resultados de las tres consultas sobre relaciones

--- CONSULTA 1 ---	--- CONSULTA 2 ---
Personas que trabajan en TechCorp <pre>[{ "persona": "Ana", "empresa": "TechCorp" }, { "persona": "Carlos", "empresa": "TechCorp" }, { "persona": "Sofia", "empresa": "TechCorp" }]</pre>	Personas que viven en Medellin <pre>[{ "persona": "Ana", "ciudad": "Medellin" }, { "persona": "Laura", "ciudad": "Medellin" }, { "persona": "Sofia", "ciudad": "Medellin" }]</pre>

```

--- CONSULTA 3 ---
Recorrido de dos niveles desde Ana
[
  {
    "origen": "Ana",
    "contacto1": "Carlos",
    "contacto2": "Miguel"
  },
  {
    "origen": "Ana",
    "contacto1": "Laura",
    "contacto2": "Sofia"
  }
]

```

- **Evidencia 6 – Verificación final del grafo: 10 vértices y 18 relaciones**

```

--- VERIFICACION ---
[
  {
    "vertices": 10,
    "relaciones": 18
  }
]

```

Documente la VPC, las rutas, los grupos de seguridad y la autenticación de la opción elegida.

R/ Se utilizó la VPC por defecto de la región us-east-2, con identificador vpc-066a250055233310a. El clúster Neptune quedó asociado al subnet group por defecto de esa VPC y a varias subredes activas dentro de la región.

Para permitir el acceso público al endpoint de Neptune, las subredes utilizadas deben disponer de una ruta de salida hacia Internet mediante un Internet Gateway, normalmente con una ruta 0.0.0.0/0. En este caso se utilizó la VPC por defecto, que ya dispone de conectividad pública adecuada.

Se creó el Security Group neptune-punto4-sg. La regla de entrada fue restringida para permitir únicamente tráfico TCP por el puerto 8182 desde la IP pública del equipo local.

Se habilitó IAM Database Authentication en el clúster. Para acceder desde el equipo local se utilizó el usuario IAM user_cli, al que se agregó la política TallerAWSNeptuneDataAccess con permisos para consultar y modificar datos mediante Neptune. Las solicitudes se autenticaron usando las credenciales AWS configuradas en el perfil user_cli, sin utilizar usuario y contraseña de base de datos.

¿Es necesario este proceso con EC2? ¿Existe otra manera de realizar dicho proceso?

R/ No, en nuestro caso no fue necesario utilizar una instancia EC2. Como el clúster de Neptune quedó configurado con Public connectivity = Yes y con IAM Database Authentication, pudimos conectarnos directamente desde el computador local al endpoint público por el puerto 8182. La alternativa clásica es utilizar una EC2 dentro de la misma VPC y establecer un túnel SSH desde el equipo local hacia esa instancia. Este método permite mantener Neptune privado, pero añade complejidad y costo por la EC2.

¿Cuánto cuestan el cómputo y el almacenamiento?

R/ En nuestro caso usamos Neptune Serverless en us-east-2. El cómputo se cobra según las NCU (Neptune Capacity Units) utilizadas, facturadas por segundo. Una NCU equivale aproximadamente a 2 GB de memoria con capacidad asociada de CPU y red. El almacenamiento se cobra aparte por los GB utilizados al mes; con Neptune Standard también se cobran las operaciones de I/O por solicitud.

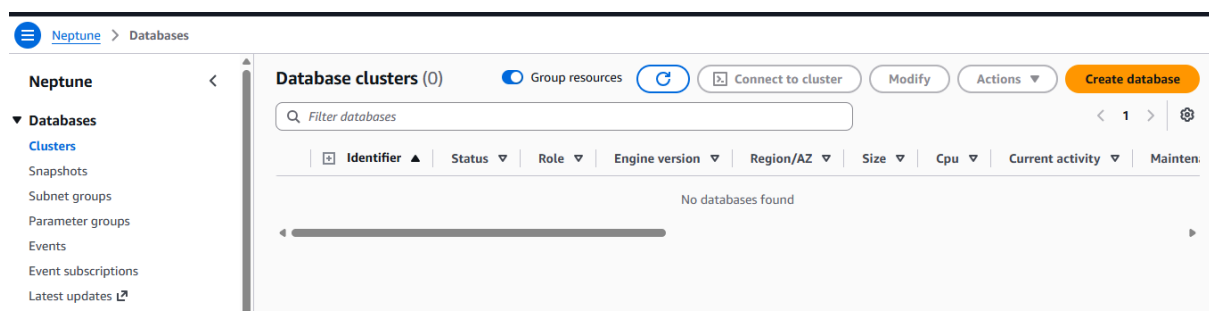
¿Su método de conexión añade una instancia EC2?

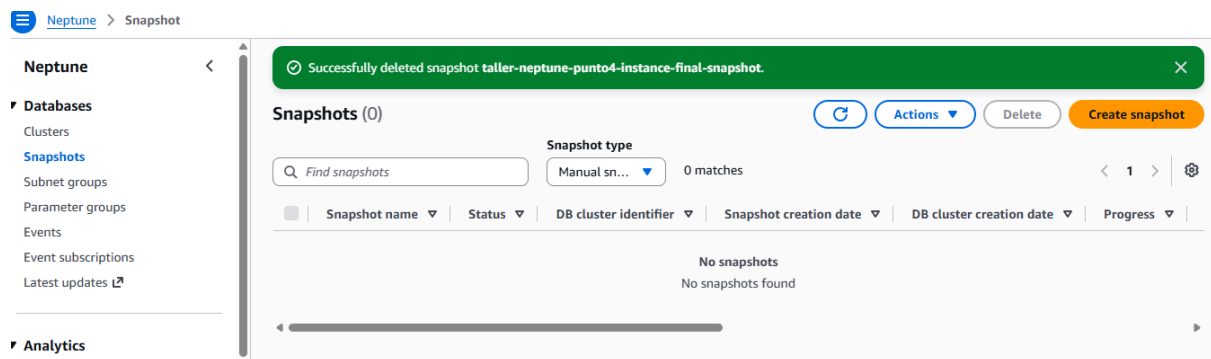
R/ No. Utilizamos un endpoint público de Neptune con autenticación IAM, por lo que no fue necesario crear una EC2 ni asumir su costo adicional.

¿Qué se sigue cobrando mientras no ejecuta consultas?

R/ Aunque no se estén ejecutando consultas, el clúster Serverless puede mantener su capacidad mínima configurada, por lo que puede seguir existiendo costo de cómputo, además del almacenamiento de los datos, backups y posibles operaciones de I/O o transferencia.

- Eliminación de los recursos de Amazon Neptune





5. Amazon Redshift

Investigue qué es Amazon Redshift, para qué tipo de análisis se utiliza y cuáles son sus principales componentes.

R/ Amazon Redshift es el data warehouse de AWS: una base de datos relacional diseñada para análisis sobre grandes volúmenes de datos, no para registrar transacciones una por una. Guarda la información por columnas en lugar de por filas. Así, cuando una consulta solo necesita dos o tres columnas de una tabla enorme, lee únicamente esas, y los datos se comprimen mucho mejor. Además, reparte el trabajo entre varios nodos que procesan en paralelo (arquitectura MPP, *Massively Parallel Processing*).

Se usa para análisis OLAP: tableros de indicadores, reportes de ventas históricas, agregaciones sobre millones de filas, análisis de tendencias y consultas que cruzan varias tablas grandes. También puede consultar archivos que están directamente en S3 mediante Redshift Spectrum.

Sus principales componentes son:

- **Nodo líder y nodos de cómputo** (en la versión aprovisionada): el nodo líder recibe las consultas, arma el plan de ejecución y lo reparte entre los nodos de cómputo, que a su vez se dividen en *slices* que trabajan en paralelo. Hoy AWS ofrece nodos RG (próxima generación), RA3 (generación actual) y DC2 (generación anterior).
- **Namespace y workgroup** (en Redshift Serverless, la versión que usamos): el namespace agrupa lo relacionado con los datos (bases de datos, esquemas, tablas, usuarios, contraseña y roles IAM), y el workgroup agrupa lo relacionado con el cómputo (capacidad en RPU, subredes, grupos de seguridad y endpoint). Cada RPU (*Redshift Processing Unit*) aporta 16 GB de memoria.
- **Redshift Managed Storage (RMS)**: el almacenamiento administrado donde quedan las tablas. Usa discos SSD locales para los datos más consultados y S3 para el almacenamiento duradero, y se cobra aparte del cómputo.
- **Claves de distribución y de ordenamiento**: deciden cómo se reparten las filas entre los nodos (DISTKEY, DISTSTYLE ALL) y en qué orden se guardan (SORTKEY). En nuestras tablas, clientes se replica completa en todos los nodos por ser pequeña, y

pedidos se distribuye por `id_cliente` y se ordena por fecha, para que el *join* y los filtros por rango de fechas sean más eficientes.

- **Herramientas de acceso:** Query Editor v2 en la consola, la API de datos de Redshift para ejecutar SQL por HTTPS sin manejar conexiones, y los controladores JDBC/ODBC para clientes SQL.
- **Comando COPY:** la forma recomendada de cargar datos de forma masiva desde S3, usando un rol IAM que le da permiso a Redshift de leer los archivos.

Compare brevemente su propósito con el de Amazon RDS e indique un caso en el que elegiría cada servicio.

R/

Aspecto	Amazon RDS	Amazon Redshift
Tipo de carga	OLTP: muchas transacciones pequeñas (insertar, actualizar, consultar un registro)	OLAP: pocas consultas grandes que recorren y agregan millones de filas
Almacenamiento	Por filas	Por columnas, comprimido
Escala	Un servidor principal con réplicas de lectura	Procesamiento paralelo en varios nodos
Restricciones	Las llaves primarias y foráneas se hacen cumplir	Las llaves primarias y foráneas son informativas (ayudan al optimizador, pero no se validan)
Motores	PostgreSQL, MySQL, MariaDB, Oracle, SQL Server	Motor propio compatible con PostgreSQL

Elegiríamos **RDS** para la base de datos de una tienda en línea, donde cada compra inserta un pedido, descuenta inventario y actualiza el estado del envío en tiempo real: son operaciones pequeñas, muy frecuentes y que necesitan integridad. Elegiríamos **Redshift** para el área de analítica de esa misma tienda: cargar todos los pedidos de los últimos años y responder preguntas como cuánto se vendió por ciudad y categoría cada mes, o qué clientes concentran más ingresos. Esas consultas recorren todo el histórico, y en RDS afectarían el rendimiento de la operación diaria. Un detalle que notamos en la práctica es que, aunque definimos una llave foránea entre pedidos y clientes, Redshift no la valida. Esto refleja que su prioridad es la velocidad de análisis y no la integridad transaccional.

Despliegue Redshift con la configuración de menor costo adecuada para el taller. Investigue las opciones Serverless y aprovisionada, y justifique su elección según la región y el tiempo previsto de uso.

R/ Trabajamos en us-east-2 (Ohio) y comparamos las dos modalidades con los precios publicados por AWS para esa región:

Modalidad	Unidad de cobro	Precio en Ohio	Cuándo se cobra	Costo para la práctica
Aprovisionada dc2.large (generación anterior, la más barata)	Hora de nodo	USD 0,25 por hora	Todo el tiempo que el clúster esté encendido	≈ USD 0,50 por 2 horas
Aprovisionada rg.large (próxima generación, la que AWS recomienda)	Hora de nodo	USD 0,3801 por hora	Todo el tiempo que el clúster esté encendido	≈ USD 0,76 por 2 horas
Aprovisionada ra3.large (generación actual)	Hora de nodo	USD 0,543 por hora	Todo el tiempo que el clúster esté encendido	≈ USD 1,09 por 2 horas
Serverless con 4 RPU (mínimo)	RPU-hora, por segundo, con mínimo de 60 s	USD 0,36 por RPU-hora (USD 1,44 por hora activa)	Solo mientras se ejecutan consultas	≈ USD 0,10 medidos (960 RPU-segundos)

Elegimos **Redshift Serverless con capacidad base de 4 RPU**, que es el mínimo permitido. Aunque una hora activa de Serverless (USD 1,44) cuesta más que una hora del nodo aprovisionado más barato (USD 0,25), en nuestra práctica el cómputo real fue de apenas unos minutos: crear las tablas, cargar dos archivos pequeños y ejecutar tres consultas. La mayor parte del tiempo la pasamos configurando permisos, leyendo resultados y tomando evidencias. En ese tiempo Serverless no cobra cómputo, mientras que un clúster aprovisionado sí, desde que se crea hasta que se elimina. Para que la opción aprovisionada fuera más barata, el workgroup tendría que estar activo más de unos 21 minutos por cada hora de trabajo ($0,25 \div 1,44 \times 60$), algo que en un taller no ocurre. Además, AWS ofrece USD 300 de crédito durante 90 días a las cuentas que nunca han usado Redshift Serverless.

El despliegue lo hicimos desde AWS CloudShell con el usuario `juan_admin`:

- Creamos el **namespace** taller-ns, con la base de datos tienda, el usuario administrador adminrs y la contraseña administrada por Secrets Manager.
- Creamos el **workgroup** taller-wg, con 4 RPU de capacidad base, en tres subredes de la VPC por defecto (us-east-2a, us-east-2b y us-east-2c) y **sin acceso público**.
- Para no llevarnos sorpresas, configuramos un **límite de uso** de 2 RPU-horas diarias con la acción deactivate. Si el workgroup consume más de eso en un día, se desactiva

automáticamente. Eso equivale a unos 30 minutos activos a 4 RPU, es decir, un máximo de USD 0,72 por día.

La modalidad aprovisionada conviene cuando hay consultas constantes durante muchas horas al día, porque su tarifa por hora es menor, se puede pausar cuando no se usa y se puede abaratar más con nodos reservados. Para cargas cortas e intermitentes como un taller, Serverless resulta más económico.

Configure la contraseña de administración mediante AWS Secrets Manager.

R/ Al crear el namespace usamos la opción `--manage-admin-password`. Con ella, Redshift generó la contraseña del usuario administrador `adminrs` y la guardó en un secreto de Secrets Manager cuyo nombre empieza por `redshift!`, sin que nosotros la escribiéramos nunca. El programa no lee directamente la contraseña: le pasa a la API de datos el ARN del secreto (`SecretArn`), y es la API la que lo usa para autenticarse en Redshift. Por eso el usuario IAM del programa, `app_redshift`, necesita el permiso `secretsmanager:GetSecretValue` sobre ese secreto, aunque el código nunca vea su valor. Usamos el mismo secreto para conectarnos desde Query Editor v2. Cuando eliminamos el namespace, el secreto se eliminó con él.

Prepare un conjunto de datos con al menos dos tablas relacionadas. Desarrolle un programa en Python con Boto3 que suba los archivos a S3, cree las tablas en Redshift, cargue los datos mediante COPY y ejecute consultas con filtro, unión y agregación. Use la API de datos de Redshift para enviar las instrucciones SQL, comprobar que terminaron y recuperar sus resultados. Configure también el rol de IAM que permite a Redshift leer los archivos de S3.

R/ Preparamos dos tablas en formato CSV:

- **clientes:** 12 registros con `id_cliente`, nombre, ciudad, segmento (Persona o Empresa) y `fecha_registro`. Los clientes son de Medellín, Bogotá, Cali, Pereira, Manizales y Barranquilla.
- **pedidos:** 40 registros entre enero y junio de 2026, con `id_pedido`, `id_cliente`, `fecha_pedido`, categoría, cantidad y `valor_total` en pesos colombianos.

Las tablas se relacionan por `id_cliente`: cada pedido pertenece a un cliente, y un cliente puede tener muchos pedidos.

Rol IAM para Redshift. Creamos el rol `RedshiftLecturaS3Punto5` con dos políticas:

- **Política de confianza:** permite que los servicios `redshift.amazonaws.com` y `redshift-serverless.amazonaws.com` asuman el rol.
- **Política de permisos:** permite únicamente `s3:ListBucket` sobre nuestro bucket y `s3:GetObject` sobre la carpeta `punto5/`.

Asociamos el rol al namespace y lo marcamos como rol por defecto. Así, en el COPY basta con escribir `IAM_ROLE default`, sin poner el ARN del rol en el código.

Programa en Python. El programa `redshift_carga_consultas.py` usa el perfil `app-redshift` y hace todo el proceso con `Boto3`:

1. Sube los dos archivos a `s3://<bucket>/punto5/` con el cliente de S3.
2. Envía cada instrucción SQL con `redshift_data.execute_statement`, indicando el workgroup `taller-wg`, la base de datos `tienda` y el ARN del secreto.
3. Como la API de datos es asíncrona y responde de inmediato con un identificador, el programa consulta cada segundo con `describe_statement` hasta que el estado sea `FINISHED`. Si es `FAILED` o `ABORTED`, muestra el error y se detiene.
4. Cuando la instrucción devuelve filas, las recupera con `get_statement_result` y las imprime con sus columnas.

Permisos del programa. El usuario `app_redshift` tiene una política de mínimo privilegio:

Permiso	Alcance
<code>s3:PutObject</code>	Solo la carpeta <code>punto5/</code> del bucket
<code>redshift-data:ExecuteStatement</code>	Solo nuestro workgroup
<code>redshift-data:DescribeStatement</code> y <code>GetStatementResult</code>	*, porque estas acciones no admiten restringirse por recurso; la API de datos solo deja ver las sentencias del mismo usuario
<code>secretsmanager:GetSecretValue</code>	Solo el secreto del namespace

Instrucciones de carga. Las tablas se crean con `CREATE TABLE`, con `DISTSTYLE ALL` para clientes y `DISTKEY(id_cliente)` y `SORTKEY(fecha_pedido)` para pedidos. Los datos se cargan con:

```
sql
COPY pedidos FROM 's3://<bucket>/punto5/pedidos.csv'
IAM_ROLE default FORMAT AS CSV IGNOREHEADER 1 DATEFORMAT
'YYYY-MM-DD' REGION 'us-east-2';
IGNOREHEADER 1 salta la fila de encabezados del CSV, y DATEFORMAT indica el
formato de las fechas. Después de la carga verificamos que clientes tuviera 12 filas y pedidos
40.
```

Explique qué pregunta responde cada consulta e interprete los resultados.

R/

- **Consulta 1 – Filtro:** ¿qué pedidos del primer trimestre de 2026 superaron \$1.000.000?

```
sql
SELECT id_pedido, fecha_pedido, categoria, valor_total FROM pedidos
```

WHERE valor_total > 1000000 AND fecha_pedido BETWEEN '2026-01-01' AND '2026-03-31'
ORDER BY valor_total DESC;

id_pedido	fecha_pedido	categoria	valor_total
3	2026-01-27	Tecnología	14.700.000
14	2026-03-29	Hogar	4.005.000
4	2026-01-29	Hogar	2.085.000
5	2026-02-10	Hogar	1.700.000
11	2026-03-12	Alimentos	1.450.000
2	2026-01-18	Tecnología	1.245.000

De los 14 pedidos del trimestre, 6 superaron el millón de pesos. El más grande fue el pedido 3, de tecnología, por \$14.700.000, más de tres veces el segundo (pedido 14, de hogar, por \$4.005.000). Tres de los seis pedidos grandes fueron de la categoría hogar, lo que muestra que, aunque tecnología tuvo el pedido más alto, los pedidos grandes no dependen de una sola categoría.

- **Consulta 2 – Unión:** ¿qué compraron los clientes de Medellín y quién hizo cada pedido?

sql

SELECT p.id_pedido, p.fecha_pedido, c.nombre, c.segmento, p.categoria, p.valor_total
FROM pedidos p JOIN clientes c ON c.id_cliente = p.id_cliente
WHERE c.ciudad = 'Medellín' ORDER BY p.fecha_pedido;

id_pedido	fecha_pedido	nombre	segmento	categoria	valor_total
2	2026-01-18	Laura Gómez	Persona	Tecnología	1.245.000
4	2026-01-29	Andrés Restrepo	Persona	Hogar	2.085.000
7	2026-02-13	Andrés Restrepo	Persona	Alimentos	330.000
9	2026-02-27	Laura Gómez	Persona	Alimentos	705.000
20	2026-04-30	Andrés Restrepo	Persona	Tecnología	985.000

25	2026-05-14	Distribuidora Andina S.A.S.	Empresa	Oficina	4.980.000
29	2026-05-24	Andrés Restrepo	Persona	Alimentos	940.000
35	2026-06-07	Andrés Restrepo	Persona	Hogar	380.000
37	2026-06-10	Andrés Restrepo	Persona	Tecnología	1.405.000

La ciudad no está en la tabla de pedidos, así que fue necesario unir ambas tablas por `id_cliente`. Los clientes de Medellín hicieron 9 pedidos. Andrés Restrepo es el cliente más frecuente, con 6 pedidos que suman \$6.125.000. Sin embargo, el pedido individual más alto de la ciudad lo hizo la única empresa, Distribuidora Andina S.A.S., con una compra de oficina por \$4.980.000: casi el 38 % de lo vendido en Medellín en un solo pedido.

- **Consulta 3 – Agregación:** ¿cuánto se vendió en cada ciudad, cuántos pedidos hubo y cuál fue el valor promedio por pedido?

sql

```
SELECT c.ciudad, COUNT(*) AS pedidos, SUM(p.valor_total) AS total_ventas,
       ROUND(AVG(p.valor_total::DECIMAL(14,2))::BIGINT AS ticket_promedio
FROM pedidos p JOIN clientes c ON c.id_cliente = p.id_cliente
GROUP BY c.ciudad ORDER BY total_ventas DESC;
```

ciudad	pedidos	total_ventas	ticket_promedio
Barranquilla	6	18.090.000	3.015.000
Medellín	9	13.055.000	1.450.556
Manizales	5	9.735.000	1.947.000
Cali	7	6.840.000	977.143
Bogotá	11	6.680.000	607.273
Pereira	2	1.920.000	960.000

Barranquilla encabeza las ventas con \$18.090.000 en solo 6 pedidos, pero el 81 % de ese valor viene de un único pedido de

14.700.000 de Logística Caribe S. A. S. Sin ese pedido, quedaría por debajo de Medellín, Manizales y Cali (607.273), porque son compras pequeñas y frecuentes. En total se vendieron \$56.320.000 en 40 pedidos. Un detalle técnico: en Redshift, AVG sobre una columna entera devuelve un entero truncado, por eso convertimos el valor a DECIMAL antes de promediar y luego redondeamos.

Puede usar Query Editor v2 para inspeccionar la base de datos. Si además decide conectarse desde su computador con un cliente SQL, documente la configuración de red y del grupo de seguridad necesaria.

R/ Usamos Query Editor v2 desde la consola. La primera vez tuvimos que configurar la cuenta del editor, dejando el cifrado con la llave administrada por AWS, porque esa opción no se puede cambiar después. Luego nos conectamos al workgroup taller-wg con el mismo secreto de Secrets Manager. En el árbol de recursos verificamos que las tablas clientes y pedidos existieran en el esquema public de la base tienda, y consultamos los primeros registros de pedidos. También consultamos la vista del sistema sys_serverless_usage para medir el consumo real de cómputo, que usamos en las preguntas de costos.

No nos conectamos con un cliente SQL desde nuestro computador, porque la API de datos y Query Editor v2 funcionan a través de los servicios de AWS. Por eso el workgroup quedó **sin acceso público** y su grupo de seguridad sin ninguna regla de entrada desde internet.

Si hubiéramos querido usar un cliente como DBeaver, habría hecho falta:

- Activar el acceso público del workgroup.
- Tener las subredes con ruta 0.0.0.0/0 hacia un Internet Gateway.
- Agregar al grupo de seguridad una regla de entrada TCP en el puerto 5439 únicamente desde nuestra IP (/32).
- Conectarnos al endpoint del workgroup con SSL, con el usuario y la contraseña del secreto.

Preguntas de costos

¿Qué modalidad de Redshift cuesta menos para esta práctica?

R/ Redshift Serverless, y con mucha diferencia. Al terminar la práctica consultamos la vista del sistema sys_serverless_usage desde Query Editor v2 y obtuvimos el consumo real: el workgroup estuvo activo solo **122 segundos**, pero se facturaron **960 RPU-segundos**. Esto equivale a 240 segundos a 4 RPU, porque Redshift Serverless cobra un mínimo de 60 segundos cada vez que se activa, y en nuestra práctica se activó unas cuatro veces: con el programa Python y con las consultas de Query Editor v2. Con la tarifa de Ohio, eso da $960 \div 3.600 \times \text{USD } 0,36 = \text{USD } 0,096$, unos diez centavos de dólar.

En comparación, el nodo aprovisionado más barato en Ohio, un dc2.large de generación anterior, habría costado unos USD 0,50 encendido las dos horas de trabajo: unas cinco veces más. Un rg.large, el que AWS recomienda hoy, habría costado unos USD 0,76: casi ocho veces más. Y eso sin contar el tiempo de creación del clúster. Además, la cuenta podía usar el crédito de prueba de USD 300 de Redshift Serverless, así que el costo real fue prácticamente cero.

Lo que más nos llamó la atención fue el efecto del mínimo de 60 segundos: aunque nuestras consultas tardaban milisegundos, cada activación se cobró como un minuto completo. Con

muchas consultas pequeñas y espaciadas en el tiempo, conviene agruparlas para que entren en la misma ventana de cobro.

¿Cómo se cobran el cómputo y el almacenamiento?

R/ En Redshift Serverless se cobran por separado:

- **Cómputo:** se mide en RPU-horas, a USD 0,36 por RPU-hora en Ohio. Se cobra por segundo, con un mínimo de 60 segundos, y solo mientras hay consultas en ejecución; el tiempo de arranque no se cobra. Si la carga crece, Redshift puede escalar por encima de la capacidad base, y esos RPU adicionales se cobran a la misma tarifa.
- **Almacenamiento:** se cobra en Redshift Managed Storage a USD 0,024 por GB-mes, calculado por hora según los datos guardados, mientras el namespace exista y aunque no se ejecute ninguna consulta. Nuestras dos tablas ocupan unos pocos KB, así que el costo fue despreciable.
- **Copias de recuperación:** los puntos de recuperación automáticos de menos de 24 horas no se cobran. Las instantáneas manuales sí se cobran por los bloques de datos únicos que guardan.
- **Transferencia de datos:** mover los archivos entre S3 y Redshift dentro de la misma región, como hizo nuestro COPY, no tiene costo.

En la modalidad aprovisionada, en cambio, el cómputo se cobra por hora de nodo durante todo el tiempo que el clúster esté encendido, salvo que se pause. Con nodos RA3 o RG, el almacenamiento administrado se cobra aparte.

Concepto	Costo de la práctica
Cómputo Serverless (960 RPU-segundos medidos × USD 0,36/RPU-hora)	≈ USD 0,096 (cubierto por el crédito de prueba)
Almacenamiento administrado (KB × USD 0,024/GB-mes)	≈ USD 0,00
Puntos de recuperación (< 24 h)	USD 0,00
Transferencia S3 → Redshift (misma región)	USD 0,00
Secreto administrado (≈ 2 h × USD 0,40/mes)	≈ USD 0,001
S3 (dos archivos pequeños y pocas solicitudes)	≈ USD 0,00
Total	≈ USD 0,10 sin crédito; ≈ USD 0 con crédito de prueba

¿Qué recursos podrían seguir generando cargos después de ejecutar las consultas?

R/ Aunque el cómputo Serverless deja de cobrar cuando no hay consultas, varios recursos siguen generando cargos mientras existan:

- **El almacenamiento del namespace**, a USD 0,024 por GB-mes.
- **Las instantáneas manuales**, y los puntos de recuperación que se decida conservar más de 24 horas.
- **El secreto de Secrets Manager** con la contraseña del administrador, a USD 0,40 al mes.
- **Los archivos que queden en el bucket de S3.**
- **Cualquier herramienta que siga enviando consultas**, como un tablero que se actualiza solo o una consulta programada. Esas consultas vuelven a activar el workgroup y, por el mínimo de 60 segundos, cada activación cuesta al menos un minuto de cómputo.

En el caso aprovisionado sería peor: el clúster completo seguiría cobrando cada hora, y si se pausa, se siguen cobrando las copias de seguridad.

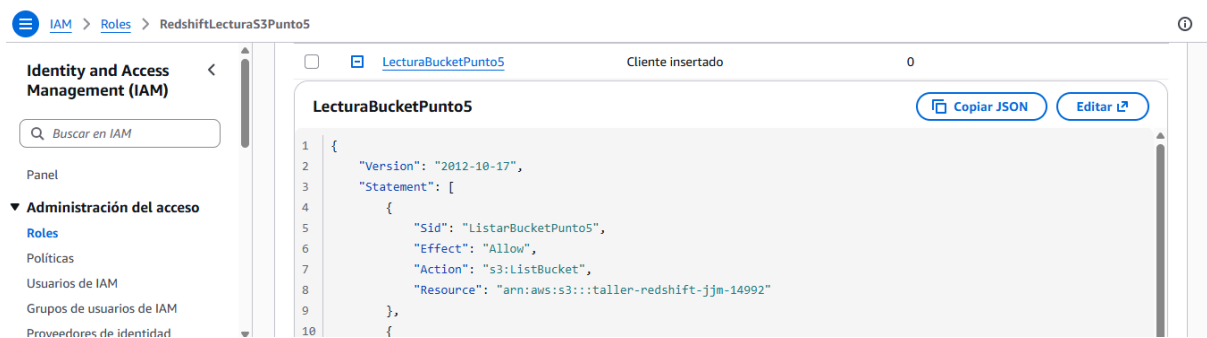
Por eso, al terminar eliminamos en orden:

1. El workgroup, que se llevó consigo el límite de uso.
2. El namespace, sin instantánea final, que eliminó también el secreto administrado.
3. El bucket con sus archivos.
4. El rol RedshiftLecturaS3Punto5.
5. El usuario IAM app_redshift con su clave de acceso.

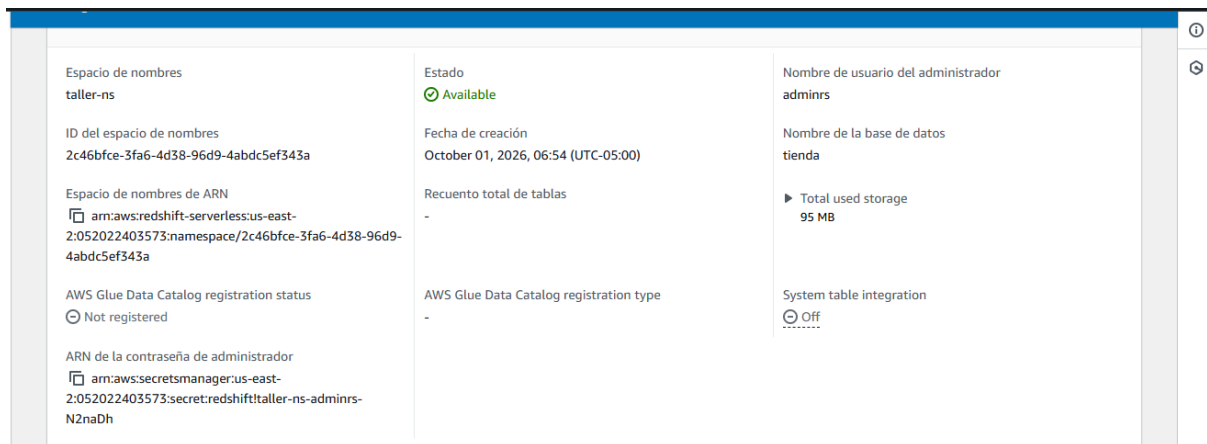
Luego verificamos que no quedaran workgroups, namespaces, instantáneas, buckets, secretos, roles ni usuarios del punto. La configuración de cuenta de Query Editor v2 la dejamos, porque no tiene costo.

Entregables:

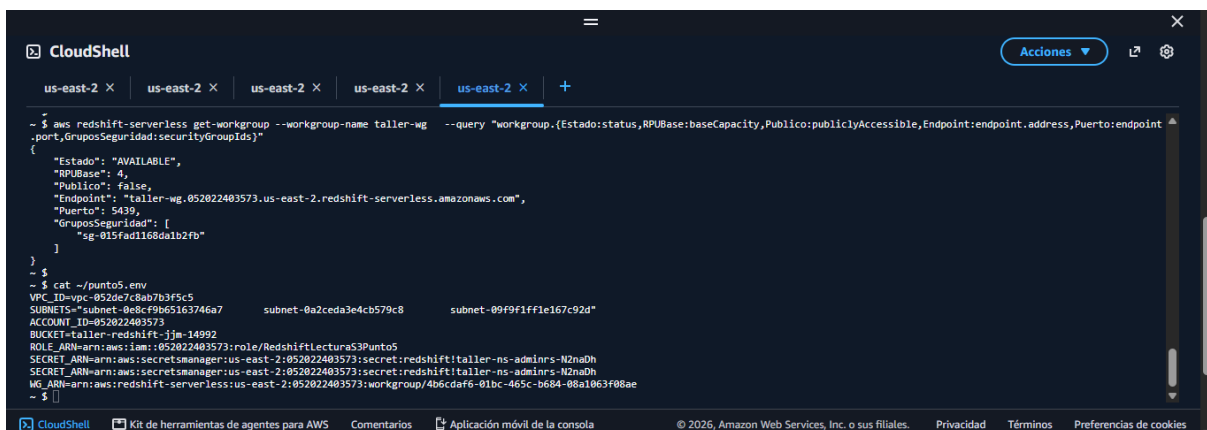
- **Conjunto de datos:** está en GitHub (punto5_redshift/datos/clientes.csv y punto5_redshift/datos/pedidos.csv).
 - **Programa Python:** está en GitHub (punto5_redshift/redshift_carga_consultas.py).
 - **Instrucciones de carga y consultas SQL:** están en GitHub (punto5_redshift/consultas.sql).
 - **Rol de Redshift y política del programa:** están en GitHub (punto5_redshift/rol_redshift_confianza.json, punto5_redshift/rol_redshift_permisos_s3.json y punto5_redshift/politica_app_redshift.json).
 - **Resultados interpretados, comparación con RDS y estimación de costos:** incluidos arriba.
- **Evidencia 1– Rol IAM para que Redshift lea S3**



- Evidencia 2 – Namespace con contraseña administrada en Secrets Manager



- Evidencia 3 – Workgroup Serverless de 4 RPU sin acceso público



- Evidencia 4 – Límite de uso del workgroup

```
aws redshift-serverless create-usage-limit \
--resource-arn $MSG_ARN \
--usage-type serverless-compute \
$ source ~/puntos.env
$ aws redshift-serverless create-usage-limit \
--resource-arn $MSG_ARN \
--usage-type serverless-compute \
--amount 2 \
--period daily \
--breach-action deactivate
{
  "usageLimit": {
    "amount": 2,
    "breachAction": "deactivate",
    "period": "daily",
    "resourceArn": "arn:aws:redshift-serverless:us-east-2:052022403573:workgroup/4b6cdf6-01bc-465c-b684-08a1063f08ae",
    "usageLimitArn": "arn:aws:redshift-serverless:us-east-2:052022403573:usageLimit/d98c366a-8170-4d8f-8f15-35cf7df33a77",
    "usageLimitId": "d98c366a-8170-4d8f-8f15-35cf7df33a77",
    "usageType": "serverless-compute"
  }
}
```

- Evidencia 5 – Política de mínimo privilegio del programa Python

```
(aws-taller) juanjo@DESKTOP-PGAPLBA:~/Taller-AWS$
(aws-taller) juanjo@DESKTOP-PGAPLBA:~/Taller-AWS$ aws configure --profile app-redshift

Tip: You can deliver temporary credentials to the AWS CLI using your AWS Console session by running the command 'aws login'.

AWS Access Key ID [None]: AKIAQYHGG4H23M4WYYOU
AWS Secret Access Key [None]: i1xfstmqcBzZMDIwKbZCZPt3rLgdvVYNk3Ukjzm
Default region name [None]: us-east-2
Default output format [None]: json

aws: [WARNING]: The file '/home/juanjo/.aws/credentials' is accessible by other users. Consider running 'chmod 600 /home/juanjo/.aws/credentials' to restrict access to only your user.
(aws-taller) juanjo@DESKTOP-PGAPLBA:~/Taller-AWS$ chmod 600 /home/juanjo/.aws/credentials
(aws-taller) juanjo@DESKTOP-PGAPLBA:~/Taller-AWS$
(aws-taller) juanjo@DESKTOP-PGAPLBA:~/Taller-AWS$ aws sts get-caller-identity --profile app-redshift
{
  "UserId": "AIDAQYHGG4H275ERP4BFS",
  "Account": "052022403573",
  "Arn": "arn:aws:iam:052022403573:user/app_redshift"
}
(aws-taller) juanjo@DESKTOP-PGAPLBA:~/Taller-AWS$
```

- Evidencia 6 – Carga a S3, COPY y consultas mediante la API de datos

```
Problemas Salida Terminal Puertos 4 Consola de depuración bash - punto5_redshift

(aws-taller) juanjo@DESKTOP-PGAPLBA:~/Taller-AWS/punto5_redshift$ python redshift_carga_consultas.py
Subido: s3://taller-redshift-jjm-14992/punto5/clientes.csv
Subido: s3://taller-redshift-jjm-14992/punto5/pedidos.csv

[FINISHED] Borrar tabla pedidos (11.85 s)

[FINISHED] Borrar tabla clientes (2.01 s)

[FINISHED] Crear tabla clientes (1.01 s)

[FINISHED] Crear tabla pedidos (1.33 s)

[FINISHED] Cargar clientes con COPY (7.44 s)

[FINISHED] Cargar pedidos con COPY (7.66 s)

[FINISHED] Verificar filas cargadas (0.85 s)
tabla | filas
pedidos | 40
clientes | 12

[FINISHED] Consulta 1 - Filtro: pedidos de más de $1.000.000 en el primer trimestre de 2026 (0.31 s)
id_pedido | fecha_pedido | categoria | valor_total
3 | 2026-01-27 | Tecnología | 14700000
14 | 2026-03-29 | Hogar | 4005000
4 | 2026-01-29 | Hogar | 2085000
5 | 2026-02-10 | Hogar | 1700000
11 | 2026-03-12 | Alimentos | 1450000
2 | 2026-01-18 | Tecnología | 1245000
```

```
Problemas Salida Terminal Puertos 4 Consola de depuración bash - punto5_redshift

(aws-taller) juanjo@DESKTOP-PGAPLBA:~/Taller-AWS/punto5_redshift$ python redshift_carga_consultas.py
id_pedido | fecha_pedido | categoria | valor_total
3 | 2026-01-27 | Tecnología | 14700000
14 | 2026-03-29 | Hogar | 4005000
4 | 2026-01-29 | Hogar | 2085000
5 | 2026-02-10 | Hogar | 1700000
11 | 2026-03-12 | Alimentos | 1450000
2 | 2026-01-18 | Tecnología | 1245000

[FINISHED] Consulta 2 - Unión: pedidos de los clientes de Medellín (0.67 s)
id_pedido | fecha_pedido | nombre | segmento | categoria | valor_total
2 | 2026-01-18 | Laura Gómez | Persona | Tecnología | 1245000
4 | 2026-01-29 | Andrés Restrepo | Persona | Hogar | 2085000
7 | 2026-02-13 | Andrés Restrepo | Persona | Alimentos | 330000
9 | 2026-02-27 | Laura Gómez | Persona | Alimentos | 705000
20 | 2026-04-30 | Andrés Restrepo | Persona | Tecnología | 985000
25 | 2026-05-14 | Distribuidora Andina S.A.S. | Empresa | Oficina | 4980000
29 | 2026-05-24 | Andrés Restrepo | Persona | Alimentos | 940000
35 | 2026-06-07 | Andrés Restrepo | Persona | Hogar | 380000
37 | 2026-06-10 | Andrés Restrepo | Persona | Tecnología | 1405000

[FINISHED] Consulta 3 - Agregación: ventas por ciudad (0.58 s)
ciudad | pedidos | total_ventas | ticket_promedio
Barranquilla | 6 | 18090000 | 3015000
Medellín | 9 | 13055000 | 1450556
Manizales | 5 | 9735000 | 1947000
Cali | 7 | 6840000 | 977143
Bogotá | 11 | 6680000 | 607273
Pereira | 2 | 1920000 | 960000
❖ (aws-taller) juanjo@DESKTOP-PGAPLBA:~/Taller-AWS/punto5_redshift$
```

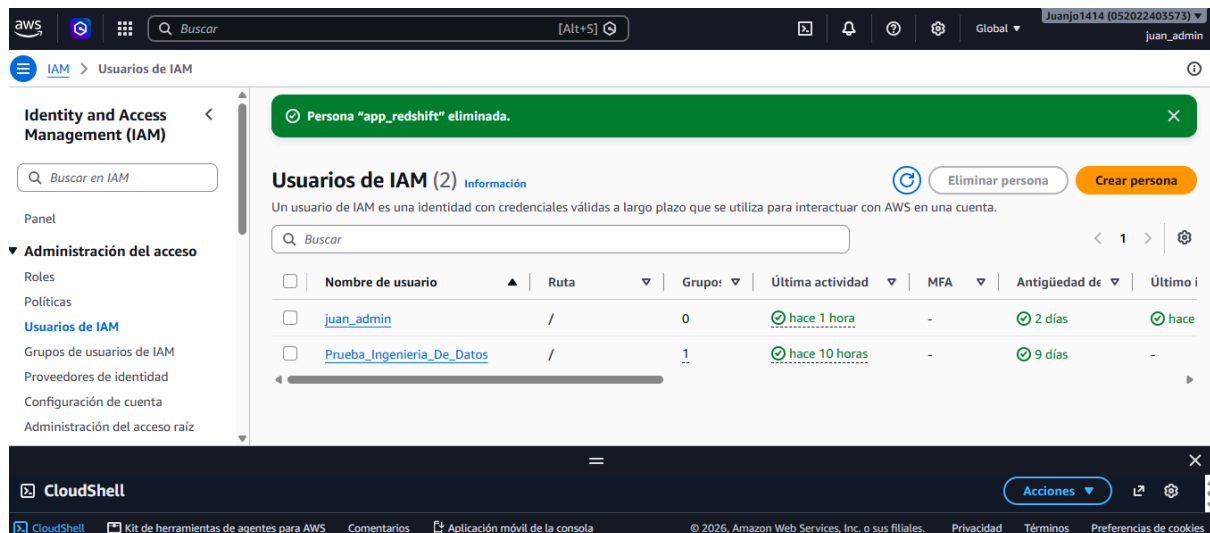
- Evidencia 7 – Base de datos inspeccionada con Query Editor v2

The screenshot displays the Amazon Redshift Query Editor v2 interface. On the left sidebar, the 'Tables' section is expanded, showing a list of tables under 'native databases (3)'. The 'pedidos' table is selected. The main editor area shows a SQL query: `SELECT * FROM pedidos ORDER BY id_pedido LIMIT 10;`. Below the query, the 'Result 1 (10)' table is displayed, showing the results of the query. The table has 5 columns: 'id_pedido', 'id cliente', 'fecha_pedido', 'categoria', and 'cantidad'. The results show 4 rows of data. The bottom status bar indicates 'Query ID: 404613', 'Elapsed time: 16210 ms', and 'Total rows: 10'.

id_pedido	id cliente	fecha_pedido	categoria	cantidad
1	4	2026-01-08	Alimentos	4
2	1	2026-01-18	Tecnología	1
3	12	2026-01-27	Tecnología	7
4	2	2026-01-29	Hogar	3

- Evidencia 8 – Eliminación de Redshift, S3 e IAM

```
us-east-2 ell
us-east-2 x us-east-2 x us-east-2 x us-east-2 x us-east-2 x us-east-2 x us-east-2 x +
delete: s3://taller-redshift-jjm-14992/punto5/pedidos.csv
delete: s3://taller-redshift-jjm-14992/punto5/clientes.csv
remove_bucket: taller-redshift-jjm-14992
~ $
~ $ aws iam delete-role-policy --role-name RedshiftLecturaS3Punto5 --policy-name LecturaBucketPunto5
~ $ aws iam create-role --role-name RedshiftLecturaS3Punto5 --policy-arn arn:aws:iam::052022403573:role/RedshiftLecturaS3Punto5"
    ],
    "kmsKeyId": "AWS_OWNED_KMS_KEY",
    "lakehouseRegistrationStatus": "NOT_REGISTERED",
    "logExports": [],
    "namespaceArn": "arn:aws:redshift-serverless:us-east-2:052022403573:namespace/2c46bfce-3fa6-4d38-96d9-4abdc5ef343a",
    "namespaceId": "2c46bfce-3fa6-4d38-96d9-4abdc5ef343a",
    "namespaceName": "taller-ns",
    "status": "DELETING"
  }
}
~ $
~ $
~ $ aws s3 rb s3://$BUCKET --force
delete: s3://taller-redshift-jjm-14992/punto5/pedidos.csv
delete: s3://taller-redshift-jjm-14992/punto5/clientes.csv
remove_bucket: taller-redshift-jjm-14992
~ $
```



- Evidencia 9 – Verificación de que no quedan recursos facturables



6. AWS Glue Data Catalog

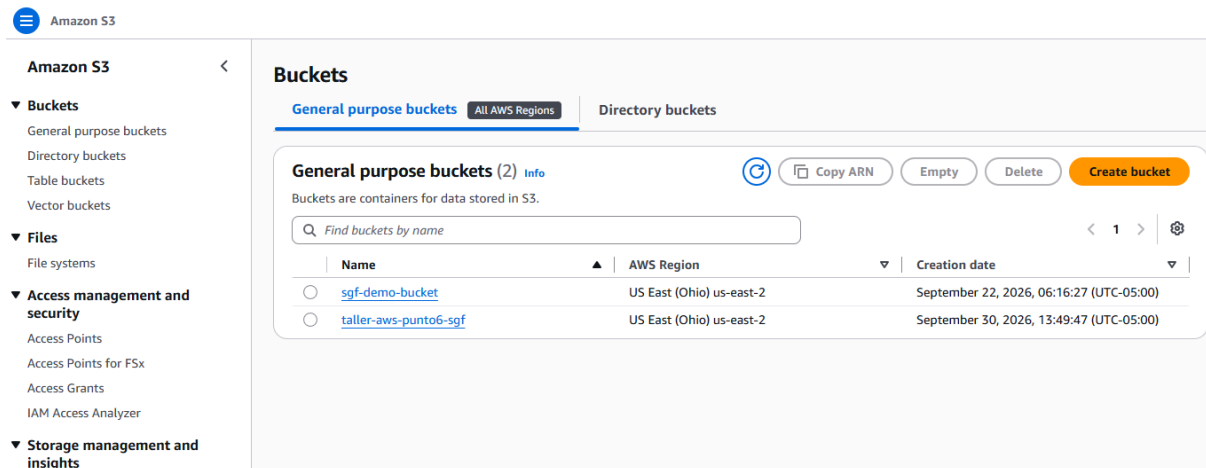
Investigue qué almacena Glue Data Catalog y distinga los archivos de datos en S3 de los metadatos registrados en el catálogo. Explique la función de las bases de datos, tablas y crawlers.

R/ AWS Glue Data Catalog almacena metadatos sobre los datos, no los datos en sí. Es decir, registra información como el nombre de las tablas, columnas, tipos de datos, ubicación de los archivos en S3, formato de los archivos y particiones. Los archivos reales, por ejemplo CSV, JSON o Parquet, permanecen almacenados en Amazon S3. Por lo tanto, la diferencia principal es que S3 almacena los datos reales, mientras que Glue Data Catalog almacena la descripción y estructura de esos datos. Además:

- **Bases de datos:** sirven para organizar lógicamente las tablas relacionadas.
- **Tablas:** representan la estructura de los datos almacenados en S3, incluyendo columnas, tipos de datos y ubicación.

- **Crawlers:** recorren los archivos almacenados en S3, detectan automáticamente su estructura y crean o actualizan las tablas correspondientes en Glue Data Catalog.

- Evidencia 1 – Creación del bucket S3 para el Punto 6



Amazon S3

Buckets

General purpose buckets | All AWS Regions | Directory buckets

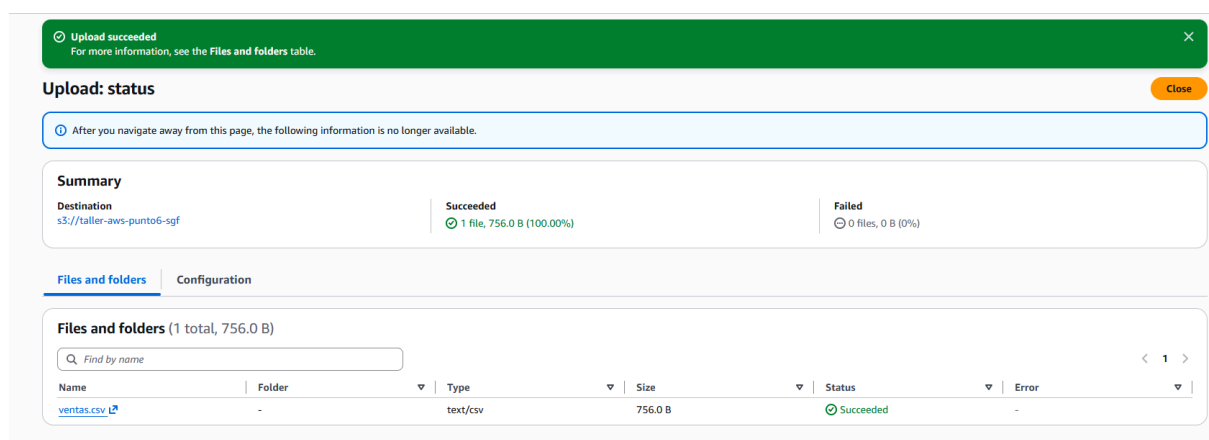
General purpose buckets (2) Info

Buckets are containers for data stored in S3.

Find buckets by name

	Name	AWS Region	Creation date
☐	sgf-demo-bucket	US East (Ohio) us-east-2	September 22, 2026, 06:16:27 (UTC-05:00)
☐	taller-aws-punto6-sgf	US East (Ohio) us-east-2	September 30, 2026, 13:49:47 (UTC-05:00)

- Evidencia 2 – Carga del archivo ventas.csv en Amazon S3



Upload: status

Upload succeeded
For more information, see the Files and folders table.

After you navigate away from this page, the following information is no longer available.

Summary

Destination: [s3://taller-aws-punto6-sgf](#)

Succeeded: 1 file, 756.0 B (100.00%)

Failed: 0 files, 0 B (0%)

Files and folders | Configuration

Files and folders (1 total, 756.0 B)

Find by name

Name	Folder	Type	Size	Status	Error
ventas.csv	-	text/csv	756.0 B	Succeeded	-

- Evidencia 3 – Creación de la base de datos en AWS Glue Data Catalog

AWS Glue > Databases

AWS now provides personalized help through Amazon Q
Click Q help to receive personalized guidance for console tasks through Q Chat. To deactivate, click [Deactivate Q help](#) to access your console settings. Once deactivated, you'll receive static help content when available.

[Launch Q help](#)

AWS Glue

- Getting started
- SageMaker Unified Studio
- ETL jobs
 - Visual ETL
 - Notebooks
 - Job run monitoring
- Data Catalog tables
- Data connections
- Workflows (orchestration)
- Zero-ETL integrations

Data Catalog

- Catalogs
- Databases**
 - Tables
- Stream schema registries
- Schemas
- Connections
- Crawlers
 - Classifiers
- Catalog settings

Data Integration and ETL

Legacy pages

Your catalog, more capabilities

Discover, govern, and trust your data in SageMaker Unified Studio with AI-powered exploration, built-in quality scores, and secure sharing across your organization

Databases (3)

A database is a set of associated table definitions, organized into a logical group.

Choose catalog
211107604485

Q Filter databases

Name	Description	Location URI	Source catalog	Created on (UTC)
dlb	-	-	211107604485	September 24, 2026 at 12:02:17
default	-	-	211107604485	September 24, 2026 at 12:02:17
taller_punto6_db	Base de datos para catalogar ventas.csv del psi	-	211107604485	September 30, 2026 at 18:55:26

AWS Glue

- Getting started
- SageMaker Unified Studio
- ETL jobs
 - Visual ETL
 - Notebooks
 - Job run monitoring
- Data Catalog tables
- Data connections
- Workflows (orchestration)
- Zero-ETL integrations

Data Catalog

- Catalogs
- Databases
- Tables
- Stream schema registries
- Schemas
- Connections
- Crawlers**

AWS Glue > Crawlers

AWS now provides personalized help through Amazon Q

Click Q help to receive personalized guidance for console tasks through Q Chat. To deactivate, click Deactivate Q help to access your console settings. Once deactivated, you'll receive static help content when available.

[Launch Q help](#)

Crawlers

A crawler connects to a data store, progresses through a prioritized list of classifiers to determine the schema for your data, and then creates metadata tables in your data catalog.

Crawlers (1) Q help
 View and manage all available crawlers.

Last updated (UTC)
September 30, 2026 at 19:16:54
Action ▾
Run
Create crawler

☐	Name	State	Schedule	Last run	Last run timestamp	Log	Table changes from last ...
☐	crawler_veritas_punto6	Ready			Succeeded	September 30, 2026 at 19:...	View log

1 created

Schema (6)

View and manage the table schema.

Q Filter schemas

< 1 >



#	Column name	Data type	Partition key	Comment
1	id	bigint	-	-
2	producto	string	-	-
3	categoria	string	-	-
4	cantidad	bigint	-	-
5	precio	bigint	-	-
6	ciudad	string	-	-

Query 1

```
1 SELECT *
2 FROM "taller_punto6_db"."taller_aws_punto6_sgf"
3 LIMIT 10;
```

Query results | Query stats

Completed Time in queue: - Run time: 351 ms Data scanned: 0.74 KB Consumed DPU: 4 DPU

Results (10)

Search rows

#	id	producto	categoría	cantidad	precio	ciudad
1	1	Laptop	Tecnología	1	3200	Medellin
2	2	Mouse	Tecnología	3	80	Bogota
3	3	Teclado	Tecnología	2	150	Cali
4	4	Silla	Oficina	1	600	Medellin
5	5	Escritorio	Oficina	1	900	Bogota
6	6	Monitor	Tecnología	2	850	Cali
7	7	Impresora	Tecnología	1	700	Medellin
8	8	Cuaderno	Papelería	5	20	Bogota
9	9	Marcadores	Papelería	4	15	Cali
10	10	Archivador	Oficina	2	120	Medellin

- Evidencia 7 – Consulta con filtro en Amazon Athena

Query 1

```
1 SELECT *
2 FROM "taller_punto6_db"."taller_aws_punto6_sgf"
3 WHERE ciudad = 'Medellin';
```

Query results | Query stats

Completed Time in queue: 36 ms Run time: 317 ms Data scanned: 0.74 KB Consumed DPU: 4 DPU

Results (7)

Search rows

#	id	producto	categoría	cantidad	precio	ciudad
1	1	Laptop	Tecnología	1	3200	Medellin
2	4	Silla	Oficina	1	600	Medellin
3	7	Impresora	Tecnología	1	700	Medellin
4	10	Archivador	Oficina	2	120	Medellin
5	13	Audifonos	Tecnología	2	250	Medellin
6	16	Webcam	Tecnología	1	300	Medellin
7	19	Soporte Monitor	Oficina	2	180	Medellin

- Evidencia 8 – Consulta con agrupación en Amazon Athena

Query 1

```
1 SELECT categoria, SUM(cantidad) AS total_unidades
2 FROM "taller_punto6_db"."taller_aws_punto6_sgf"
3 GROUP BY categoria
4 ORDER BY total_unidades DESC;
```

Query results | Query stats

Completed Time in queue: 35 ms Run time: 299 ms Data scanned: 0.74 KB Consumed DPU: 4 DPU

Results (3)

Search rows

#	categoria	total_unidades
1	Papeleria	20
2	Tecnologia	16
3	Oficina	9

- Evidencia 9 – Actualización del archivo ventas.csv en S3 (se agregan 5 registros (21-25))

Amazon S3 Buckets taller-aws-punto6-sgf

taller-aws-punto6-sgf

Objects (1)

Find objects by prefix

Name	Type	Last modified	Size	Storage class
ventas.csv	csv	September 30, 2026, 14:21:05 (UTC-05:00)	954.0 B	Standard

- Evidencia 10 – Nueva ejecución del crawler de AWS Glue

Crawler runs (2)

The list of crawler runs for this crawler.

Filter data

Filter by a date and time range

Start time (UTC)	End time (UTC)	Current/last duration	Status	DPU hours	Table changes
September 30, 2026 at 19:21:30	September 30, 2026 at 19:22:07	36 s	Completed	0.111	1 table change, 0 partition changes
September 30, 2026 at 19:01:17	September 30, 2026 at 19:01:56	39 s	Completed	0.119	1 table change, 0 partition changes

- Evidencia 11 – Verificación de nuevos registros desde Athena

Query 1

```
1 SELECT *
2 FROM "taller_punto6_db"."taller_aws_punto6_sgf"
3 WHERE id > 20
4 ORDER BY id;
```

Query results

Query stats

Completed

Time in queue: 37 ms

Run time: 374 ms

Data scanned: 0.93 KB

Consumed DPU: 4 DPU

Results (5)

Copy

Download results CSV

Search rows

#	id	producto	categoría	cantidad	precio	ciudad
1	21	Memoria USB	Tecnología	4	45	Medellín
2	22	Resma papel	Papelería	3	25	Bogotá
3	23	Silla ergonómica	Oficina	1	950	Cali
4	24	Router	Tecnología	2	380	Medellín
5	25	Notas adhesivas	Papelería	6	10	Bogotá

Compare el crawler con la creación manual de la tabla

R/ El crawler de AWS Glue automatiza el descubrimiento del esquema de los datos almacenados en S3. Detecta columnas, tipos de datos, formato y ubicación, y puede actualizar el catálogo cuando cambian los archivos. La creación manual de una tabla requiere definir explícitamente toda esa información. El crawler es más conveniente cuando los datos cambian o no se conoce bien el esquema; la creación manual ofrece mayor control cuando la estructura es estable y conocida.

Identifique si los recursos de este punto requieren alguna credencial de aplicación. Si utiliza una conexión que sí la requiere, guárdela en Secrets Manager, si trabaja únicamente con S3, Glue y Athena mediante permisos IAM, explique por qué no necesita crear otro secreto.

R/ En este punto no fue necesario crear credenciales de aplicación ni guardar secretos adicionales en AWS Secrets Manager. Se trabajó únicamente con S3, AWS Glue y Athena mediante permisos IAM. Glue accedió al bucket S3 utilizando el rol `AWSGlueServiceRole-Punto6`, mientras que Athena y la consola utilizaron los permisos asociados al usuario IAM. Como no había una base de datos externa, usuario, contraseña o conexión que requiriera autenticación propia, no fue necesario crear un secreto en Secrets Manager.

¿Qué se cobra por el crawler, el catálogo, S3 y las consultas de Athena?

R/ El crawler se cobra principalmente por el tiempo de procesamiento utilizado durante cada ejecución. En una práctica pequeña como esta, con un archivo CSV de pocos registros, el costo es muy bajo porque el tiempo de ejecución y el volumen de datos procesados son mínimos. El catálogo cobra por el almacenamiento de metadatos y por las solicitudes realizadas sobre esos metadatos. S3 cobra por el almacenamiento de los archivos y por operaciones como subir, leer o eliminar objetos. Athena cobra principalmente según la cantidad de datos examinados por cada consulta. Cuantos más datos tenga que leer una consulta, mayor será el costo.

¿Qué efecto tiene el volumen de datos examinados?

R/ El volumen de datos tiene un impacto directo sobre el costo de Athena. Una consulta que analiza muchos GB o TB cuesta más que una que examina pocos datos. Por eso es conveniente reducir la cantidad de información leída mediante filtros, particiones y formatos eficientes como Parquet.

Evidencias de Eliminación:

Eliminar el crawler

One crawler successfully deleted
The following crawler is now deleted: "crawler_ventas_punto6"

Notifications 0 0 1 1 0

Crawlers

A crawler connects to a data store, progresses through a prioritized list of classifiers to determine the schema for your data, and then creates metadata tables in your data catalog.

Crawlers (0) help

Last updated (UTC)
September 30, 2026 at 19:27:11

Action Run Create crawler

View and manage all available crawlers.

Filter crawlers

☐	Name	State	Schedule	Last run	Last run timestamp	Log	Table changes from last r...
No resources							
No resources to display.							

Eliminar la tabla del catálogo

BWS [Alt+S]

AWS Glue > Tables

Getting started
SageMaker Unified Studio
ETL jobs
Visual ETL
Notebooks
Job run monitoring

Data Catalog tables
Data connections
Workflows (orchestration)
Zero-ETL integrations

Data Catalog
Catalogs
Databases
Tables
Stream schema registries
Schemas
Connections
Crawlers
Classifiers
Catalog settings

Your catalog, more capabilities
Discover, govern, and trust your data in SageMaker Unified Studio with AI-powered exploration, built-in quality scores, and secure sharing across your organization

Set up

Tables

A table is the metadata definition that represents your data, including its schema. A table can be used as a source or target in a job definition.

Tables (0)

Last updated (UTC)
September 30, 2026 at 19:27:53

Delete Add tables using crawler Add table

View and manage all available tables.

Choose catalog
211107604485

Choose database
taller_punto6_db

Filter tables

☐	Name	Database	Location	Classification	Deprecated	View data	Data quality	Column statistics
No available tables								

Eliminar la base de datos de Glue

One database successfully deleted
The following database is now deleted: "taller_punto6_db"

Set up

Databases (2)

A database is a set of associated table definitions, organized into a logical group.

Choose catalog
211107604485

Filter databases

☐	Name	Description	Location URI	Source catalog	Created on (UTC)
☐	db	-	-	211107604485	September 24, 2026 at 12:02:17
☐	default	-	-	211107604485	September 24, 2026 at 12:02:17

Eliminar el archivo y el bucket S3

Successfully deleted objects
View details below.

Delete objects: status

Close

After you navigate away from this page, the following information is no longer available.

Summary

Source

s3://taller-aws-punto6-sgf

Successfully deleted

1 object, 954.0 B

Failed to delete

0 objects

Failed to delete

Configuration

Failed to delete (0)

Find objects by name

Name

Folder

Type

Last modified

Size

Error

No objects failed to delete.

Successfully deleted bucket "taller-aws-punto6-sgf"

Buckets

General purpose buckets

All AWS Regions

Directory buckets

General purpose buckets (1)

Info

Copy ARN

Empty

Delete

Create bucket

Find buckets by name

Name

AWS Region

Creation date

sgf-demo-bucket

US East (Ohio) us-east-2

September 22, 2026, 06:16:27 (UTC-05:00)

Account snapshot

Info

View dashboard

Updated daily

Storage Lens provides visibility into storage usage and activity trends.

External access summary

Info

Updated daily

External access findings help you identify bucket permissions that allow public access or access from other AWS accounts.

Eliminar el rol IAM opcional

Identity and Access Management (IAM)

Search IAM

Dashboard

Access Management

- Roles
- Policies
- IAM users
- IAM user groups
- Identity providers
- Account settings
- Root access management
- Temporary delegation requests

Access reports

- Access Analyzer
- Resource analysis
- Unused access
- Analyzer settings
- Policy simulator
- Credential report

Multi-account access

- Account access manager
- Organization activity
- Service control policies
- Resource control policies

Role deleted AWSGlueServiceRole-Punto6.

Roles (4)

Info

Delete

Create role

Search

Role name

Trusted entities

Last activity

AWSServiceRoleForRDS

AWS Service: rds (Service-Linked Role)

59 minutes ago

AWSServiceRoleForResourceExplorer

AWS Service: resource-explorer-2 (Service-Linked Role)

13 minutes ago

AWSServiceRoleForSupport

AWS Service: support (Service-Linked Role)

-

AWSServiceRoleForTrustedAdvisor

AWS Service: trustedadvisor (Service-Linked Role)

-

Roles Anywhere

Info

Manage

Authenticate your non AWS workloads and securely provide access to AWS services.

Access AWS from your non AWS workloads

Operate your non AWS workloads using the same authentication and authorization strategy that you use within AWS.

X.509 Standard

Use your own existing PKI infrastructure or use AWS Certificate Manager Private Certificate Authority to authenticate identities.

Temporary credentials

Use temporary credentials with ease and benefit from the enhanced security they provide.